



CMPT 413 / 713: Natural Language Processing

Recurrent Neural Networks

How to model sequences using neural networks?

Fall 2021

2021-10-14

Adapted from slides from Anoop Sarkar, Danqi Chen, Karthik Narasimhan, and Justin Johnson

(Some slides adapted from Chris Manning, Abigail See, Andrej Karpathy)

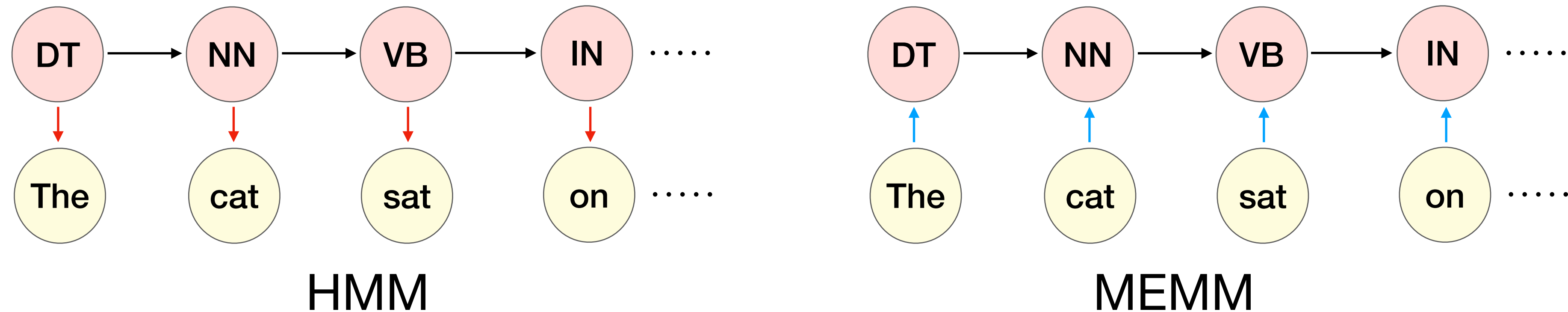
Overview

- What is a recurrent neural network (RNN)?
- Simple RNNs
- Backpropagation through time
- Applications
- Variants: Stacked RNNs, Bidirectional RNNs

What are recurrent neural
networks?

Recurrent neural networks (RNNs)

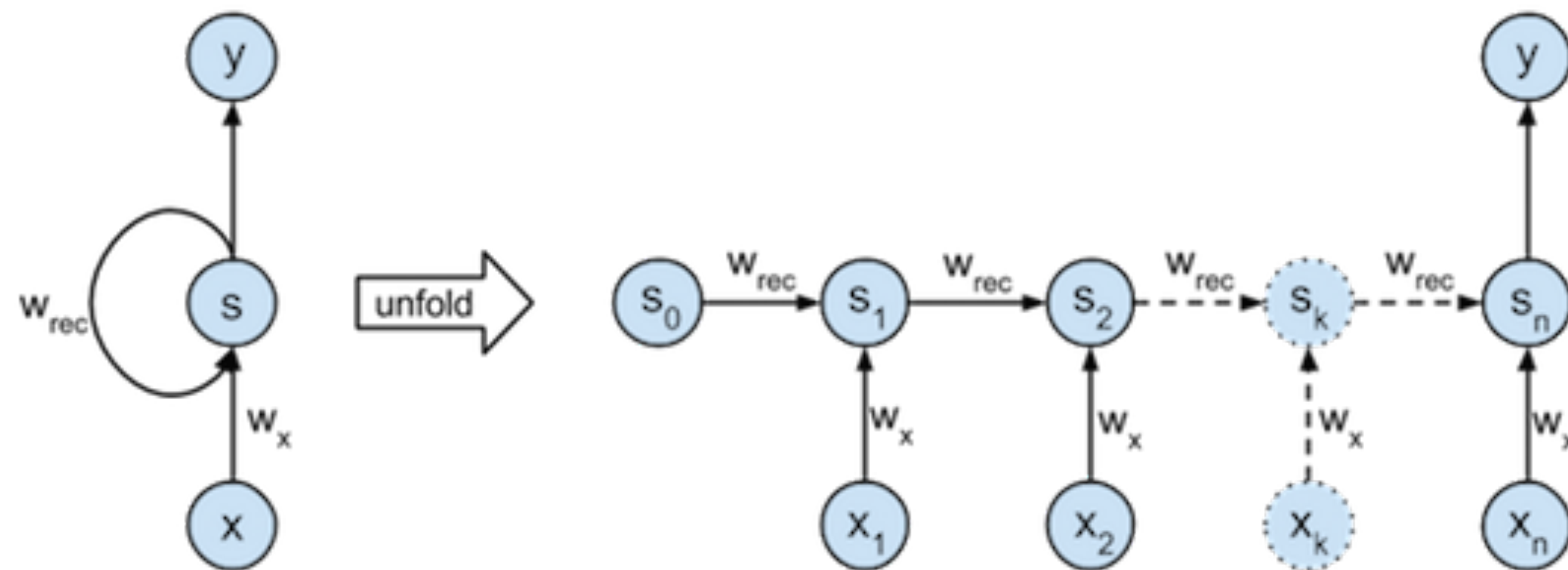
How can we model sequences using neural networks?



- Recurrent neural networks = A class of neural networks used to model sequences, allows for handling of **variable length inputs**.
- Very crucial in NLP problems (different from images) because sentences/paragraphs are variable-length, sequential inputs.

Recurrent neural networks (RNNs)

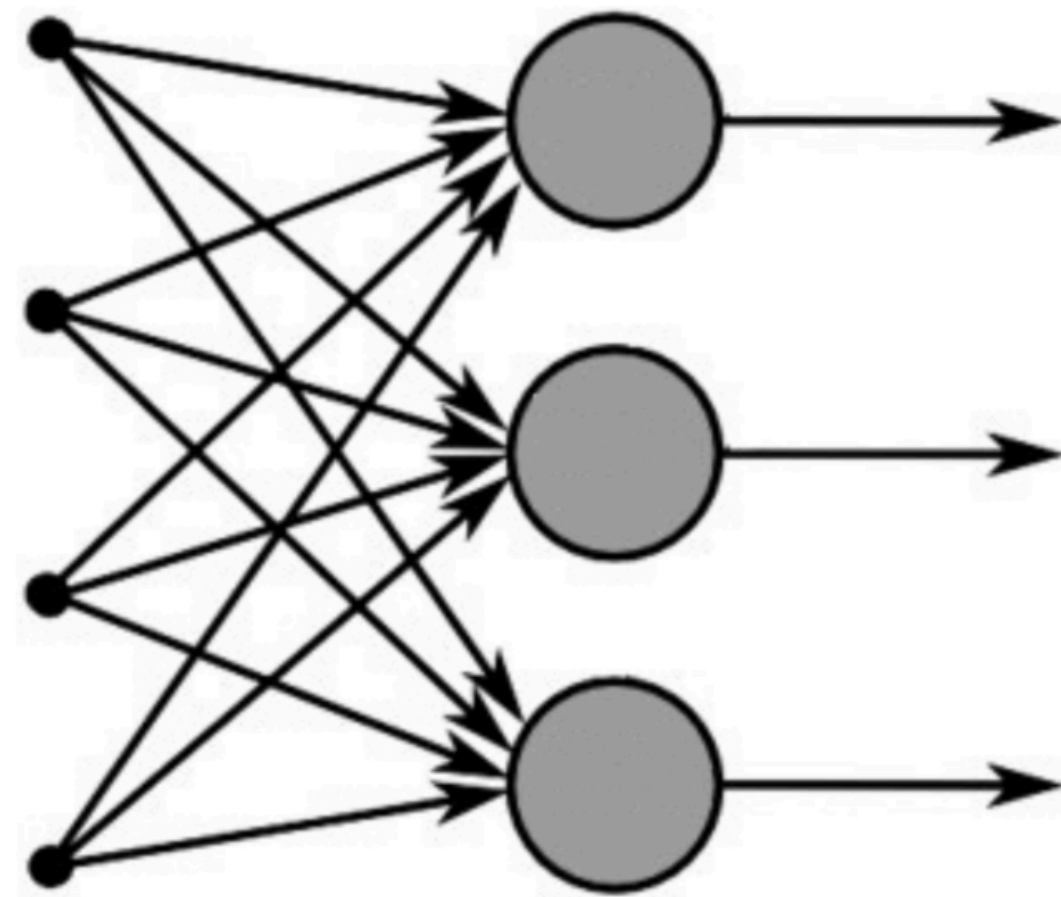
A class of neural networks allowing to handle **variable length inputs**



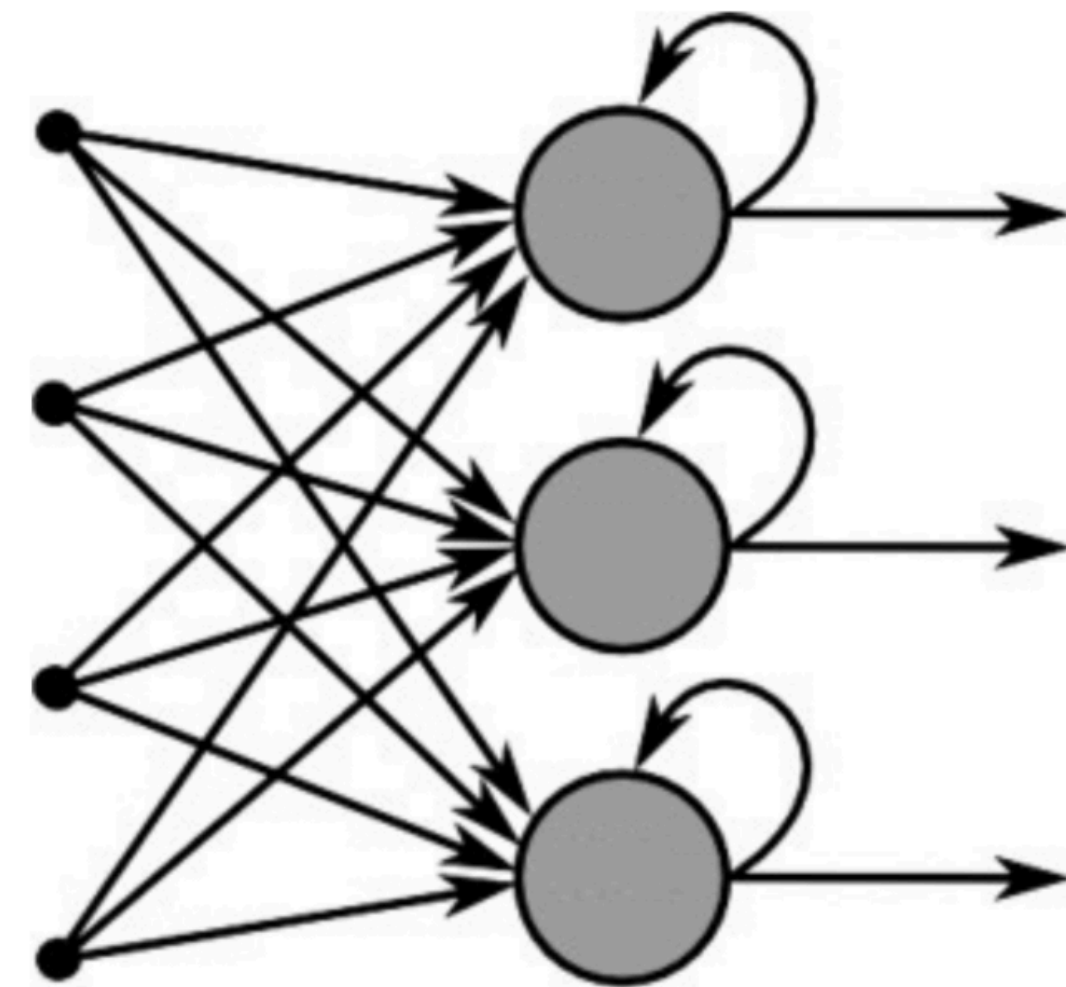
A function: $y = \text{RNN}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n) \in \mathbb{R}^d$ where $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^{d_{in}}$

Core idea: apply the same weights repeatedly at different positions

RNNs vs Feedforward NNs



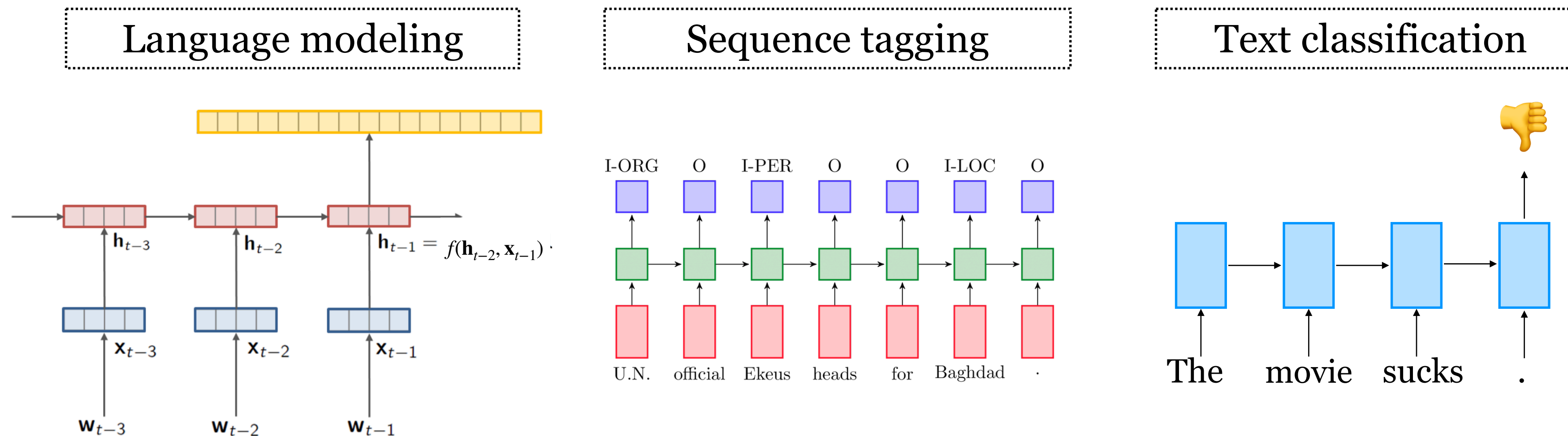
Feed-Forward Neural Network



Recurrent Neural Network

Recurrent neural networks (RNNs)

Proven to be an highly effective approach to **language modeling**, **sequence tagging** as well as **text classification** tasks:



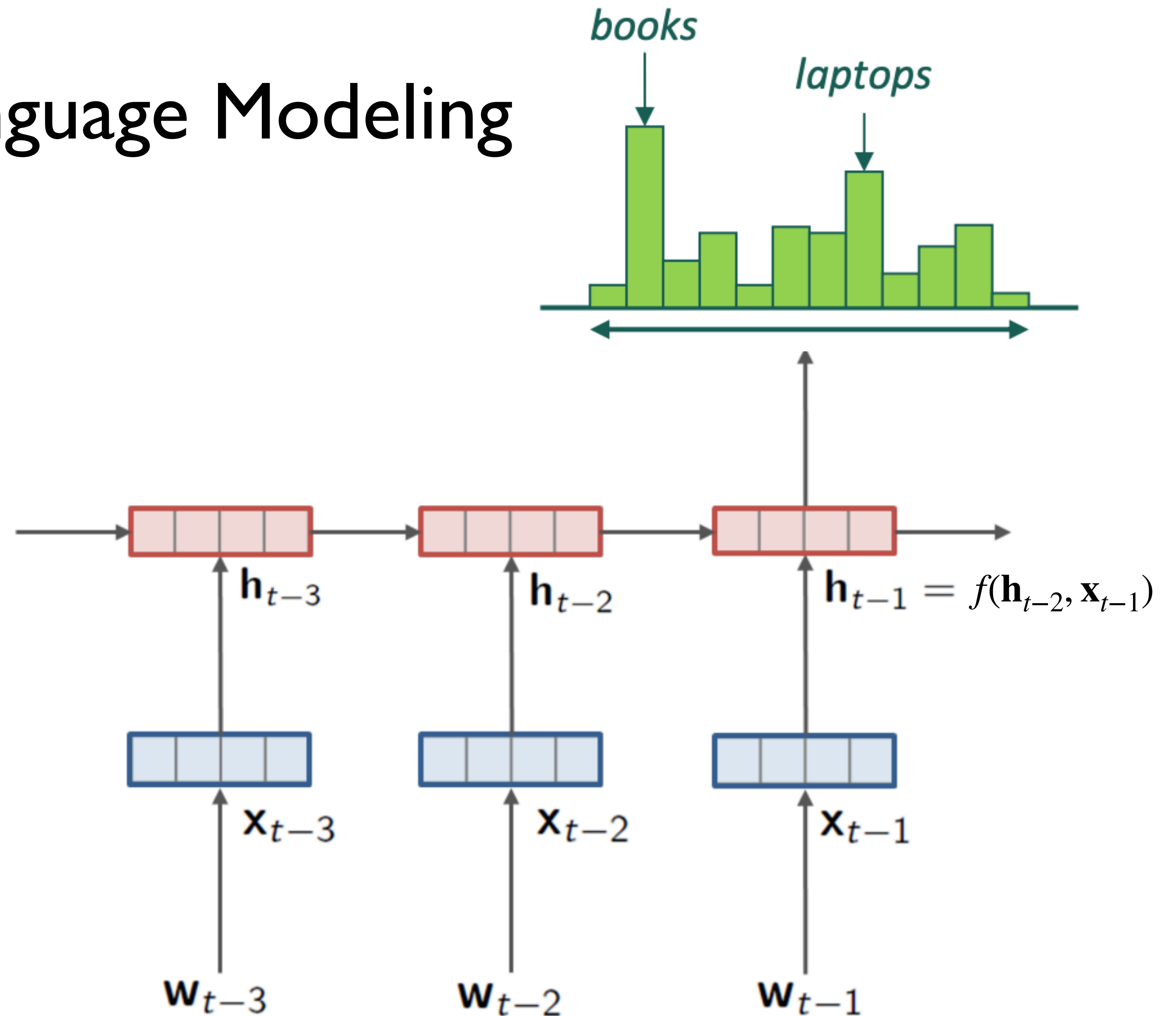
Form the basis for the modern approaches to **machine translation**, **question answering** and **dialogue**:

RNNs for Language Modeling

Use a RNN to

- capture the history of the previous words as a hidden state
- use hidden state to estimate the probability of the next word

$$P(w_t | w_1, \dots, w_{t-1}) = P(w_t | h_{t-1})$$



Recall: Language Models

- Model the probability of a sequence of words

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_1, \dots, w_{i-1})$$

- kth order Markov assumption

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

- N-grams

Bigram (1st order)

$$P(\text{mat} | \text{the cat sat on the}) \approx P(\text{mat} | \text{the})$$

Trigram (2nd order)

$$P(\text{mat} | \text{the cat sat on the}) \approx P(\text{mat} | \text{on the})$$

Issues with traditional n-grams

- Example generation from trigram model

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

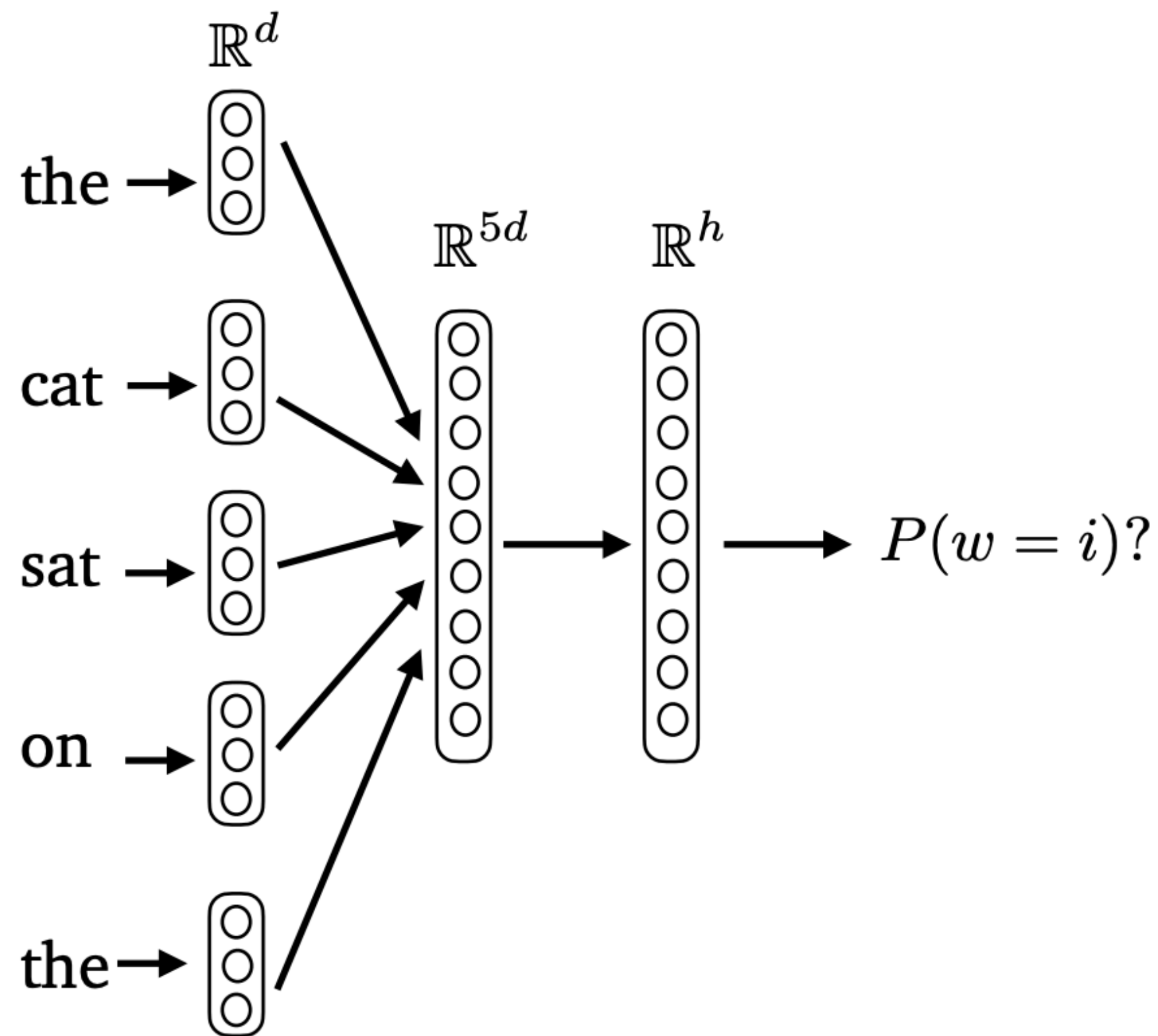
- Surprisingly grammatical!
- But **incoherent**. Need to consider longer history to model language well.

Why can't we just increase n ?

- Sparsity issues when n is large
- Model size (number of parameters) **increases** exponentially with n
 - Takes up a lot of memory to store all those probabilities

Feedforward Neural Language Model

- $P(\text{mat} \mid \text{the cat sat on the}) = ?$



- Input layer (context size $n = 5$):

$$\mathbf{e} = [\mathbf{e}_{\text{the}}; \mathbf{e}_{\text{cat}}; \mathbf{e}_{\text{sat}}; \mathbf{e}_{\text{on}}; \mathbf{e}_{\text{the}}] \in \mathbb{R}^{dn}$$

concatenate word embeddings

- Hidden layer

$$\mathbf{h} = \tanh(\mathbf{W}\mathbf{e} + \mathbf{b}_1) \in \mathbb{R}^h$$

- Output layer (softmax)

$$\mathbf{z} = \mathbf{U}\mathbf{h} + \mathbf{b}_2 \in \mathbb{R}^{|V|}$$

$$P(w = i \mid \text{context}) = \text{softmax}_i(\mathbf{z})$$

(Bengio et 2003): A Neural Probabilistic Language Model

Feedforward Neural Language Model

output distribution

$$\hat{y} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

$$\mathbf{h} = f(\mathbf{W}\mathbf{e} + \mathbf{b}_1) \quad \mathbf{W} \in \mathbb{R}^{h \times nd}$$

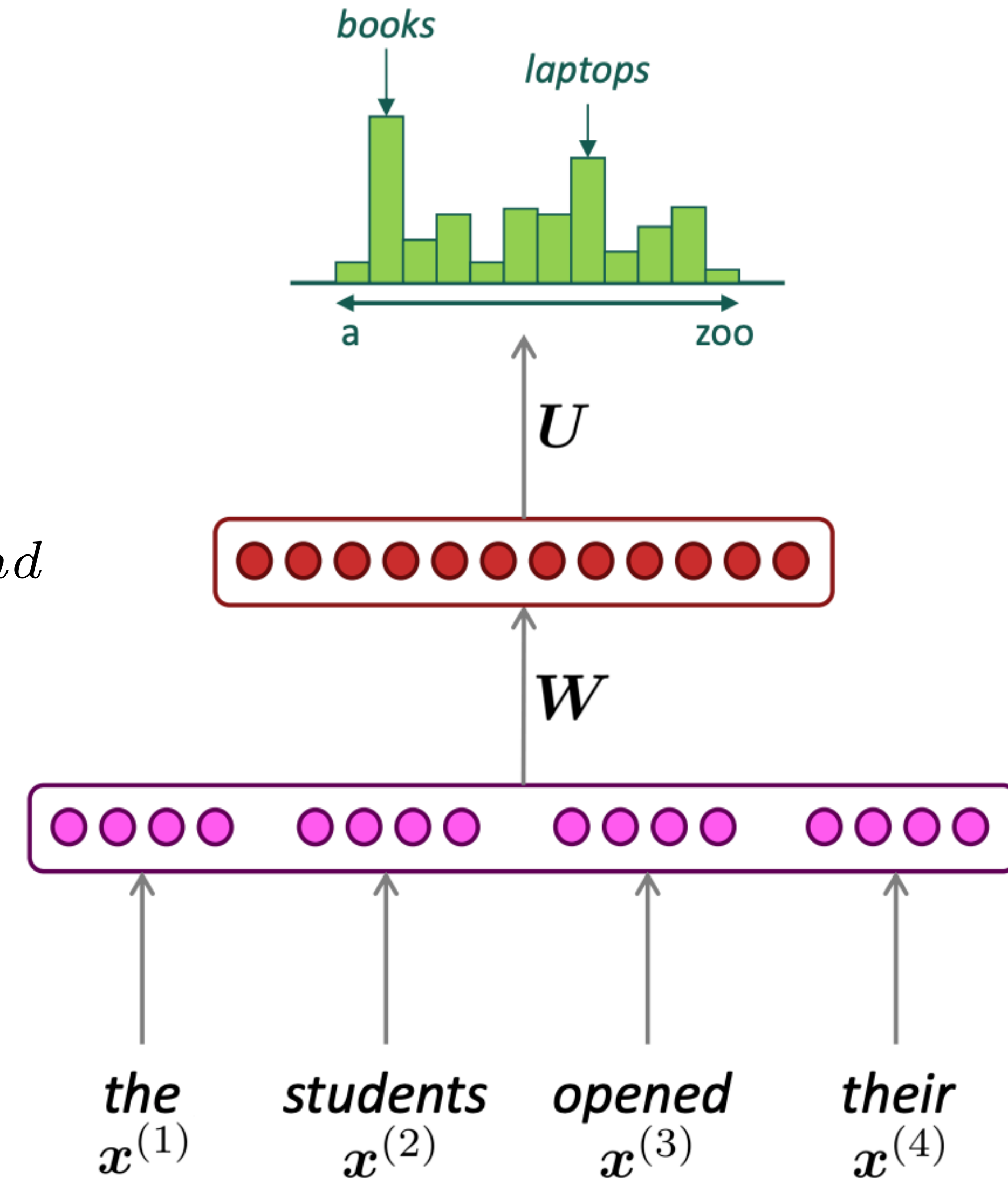
concatenated word embeddings

$$\mathbf{e} = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

Fixed window LM

words / one-hot vectors

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



Issues with fixed-window neural LM

Improvements over n-gram LMs

- No sparsity problem
- Don't need to store all observed n-grams

Remaining issues

- Fixed window is still limited in size (too small)
- Enlarging window increases parameters: $\mathbf{W} \in \mathbb{R}^{h \times nd}$
- Each word in the window is multiplied by a different set of weights
- No symmetry in how the inputs are processed

What we really want:

- Neural network to handle input sequences of arbitrary length!

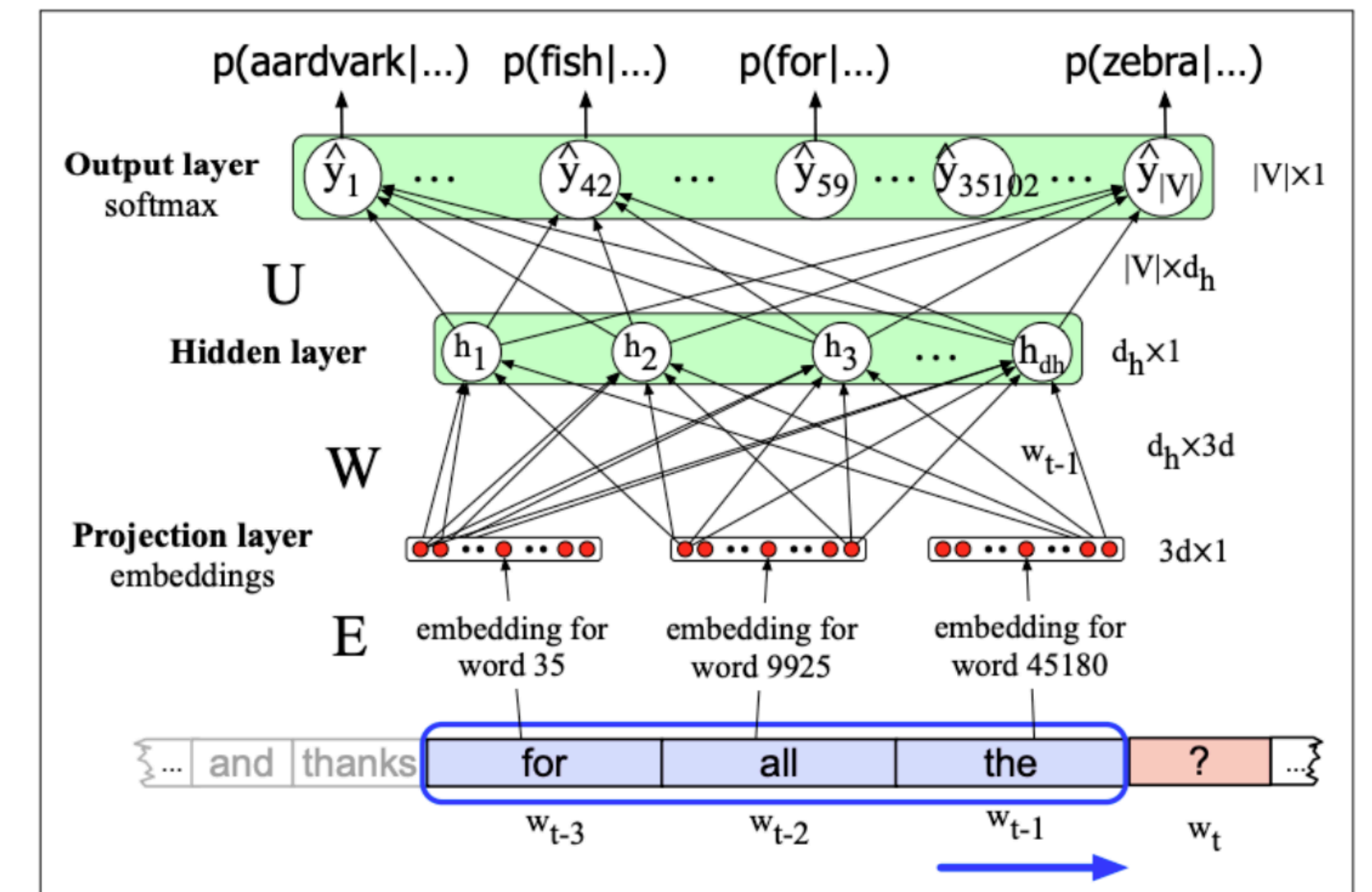


Figure 9.1 A simplified view of a feedforward neural language model moving through a text. At each time step t the network takes the 3 context words, converts each to a d -dimensional embedding, and concatenates the 3 embeddings together to get the $1 \times Nd$ unit input layer x for the network. The output of the network is a probability distribution over the vocabulary representing the model's belief with respect to each word being the next possible word.

“all the” appears in different positions of two sliding windows

Language Modeling

Predict probability of sequence of words

$$P(s) = P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{<t})$$

with n-grams

$$P(w_t | w_{<t}) \approx P(w_t | w_{t-n+1, t-1})$$

with HMMs

$$P(w_t | w_{<t}) \approx P(w_t | h_t) P(h_t | h_{t-n+1, t-1})$$

with neural networks

$$p(w_t | w_{<t}) \approx p(w_t | \phi(w_1, \dots, w_{t-1}))$$

with fixed window

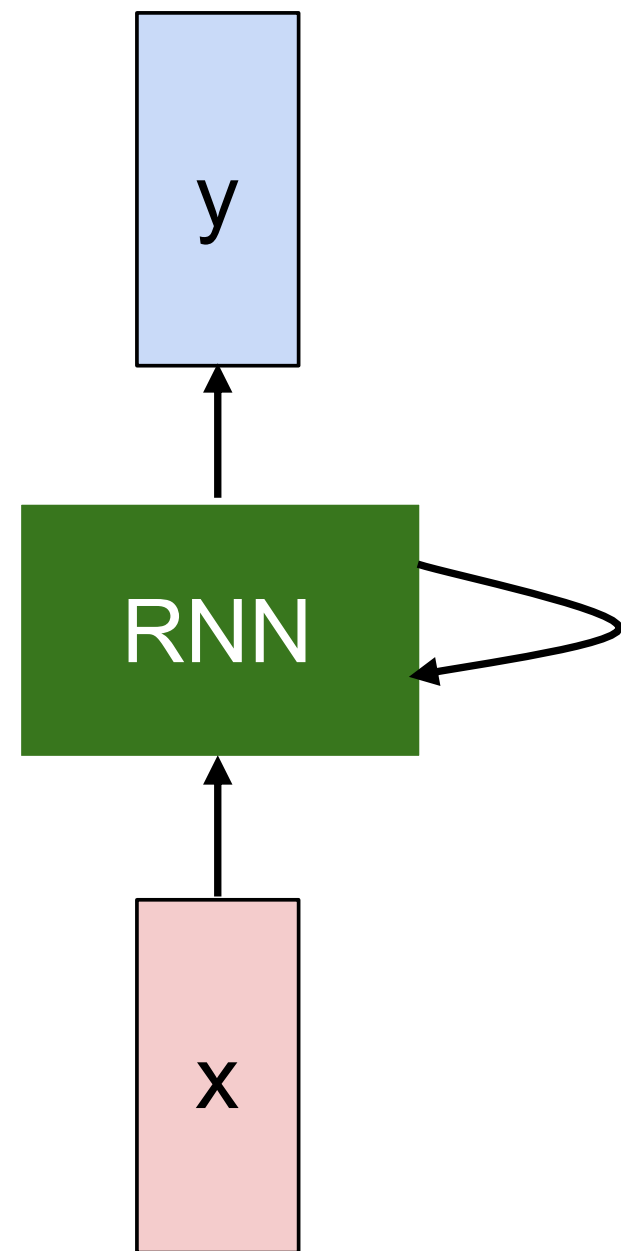
$$P(w_t | w_{<t}) \approx P(w_t | \phi(w_{t-n+1, t-1}))$$

with RNNs

$$P(w_t | w_{<t}) \approx P(w_t | \mathbf{h}_t), \mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

Simple RNNs

Components of RNN cells



$\mathbf{h}_0 \in \mathbb{R}^d$ is an initial state

function with weights W

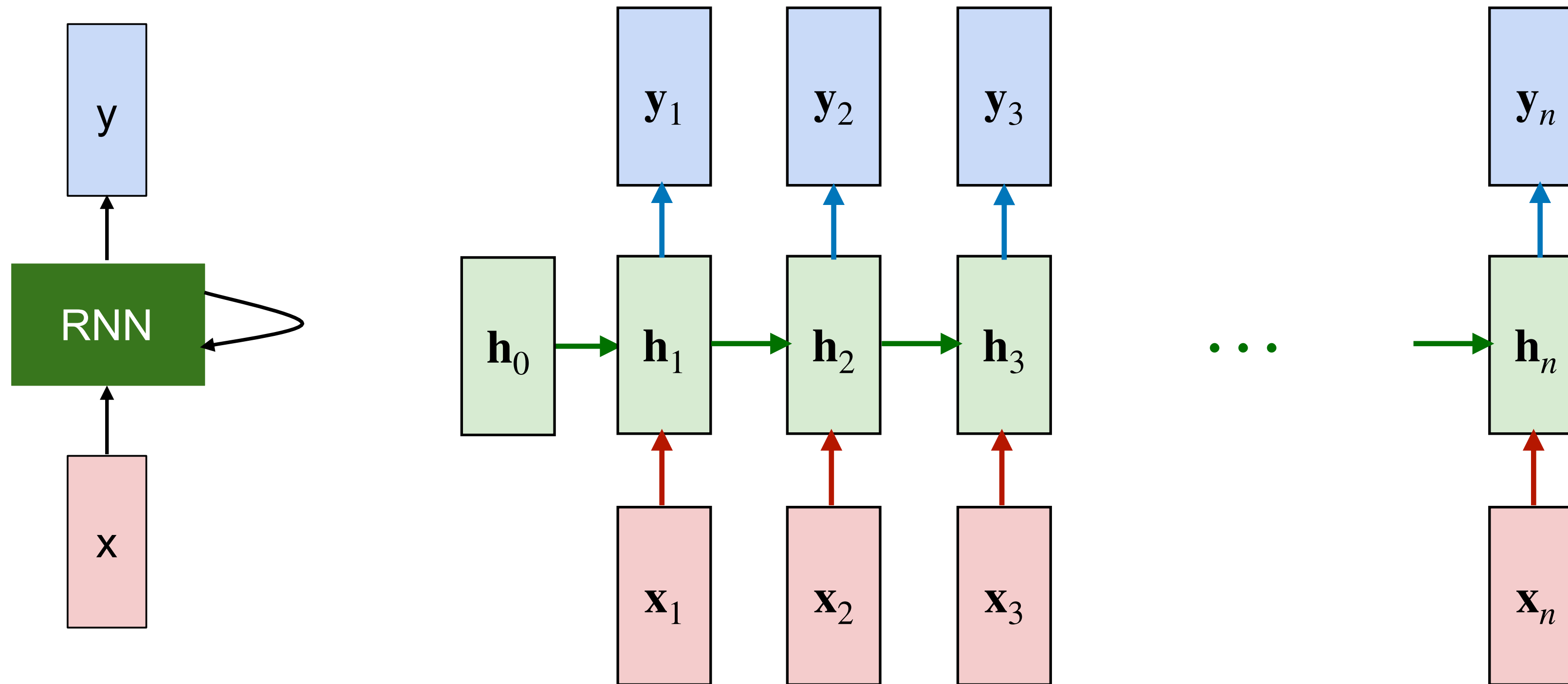
$$\mathbf{h}_t = f_{\mathbf{W}}(\mathbf{h}_{t-1}, \mathbf{x}_t) \in \mathbb{R}^d$$

new state old state input at time t

$\mathbf{h}_t$: hidden states which store information from $\mathbf{x}_1$ to $\mathbf{x}_t$

Output label for each time step: Denote $\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{W}_o \mathbf{h}_t)$, $\mathbf{W}_o \in \mathbb{R}^{|L| \times d}$

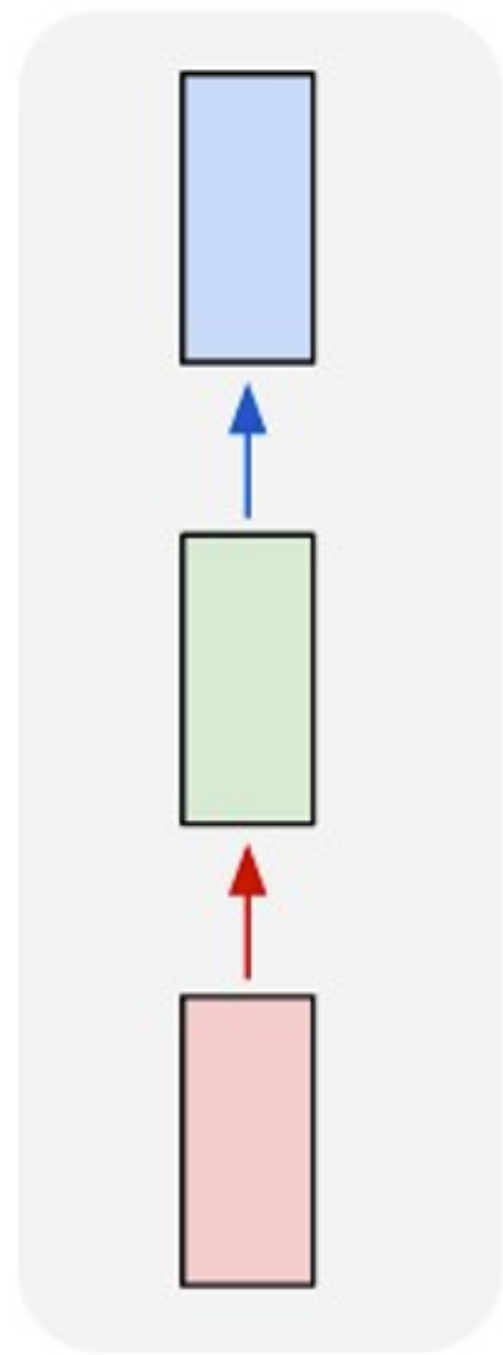
Unrolling the RNN



Structure of cell and weights are shared across time steps

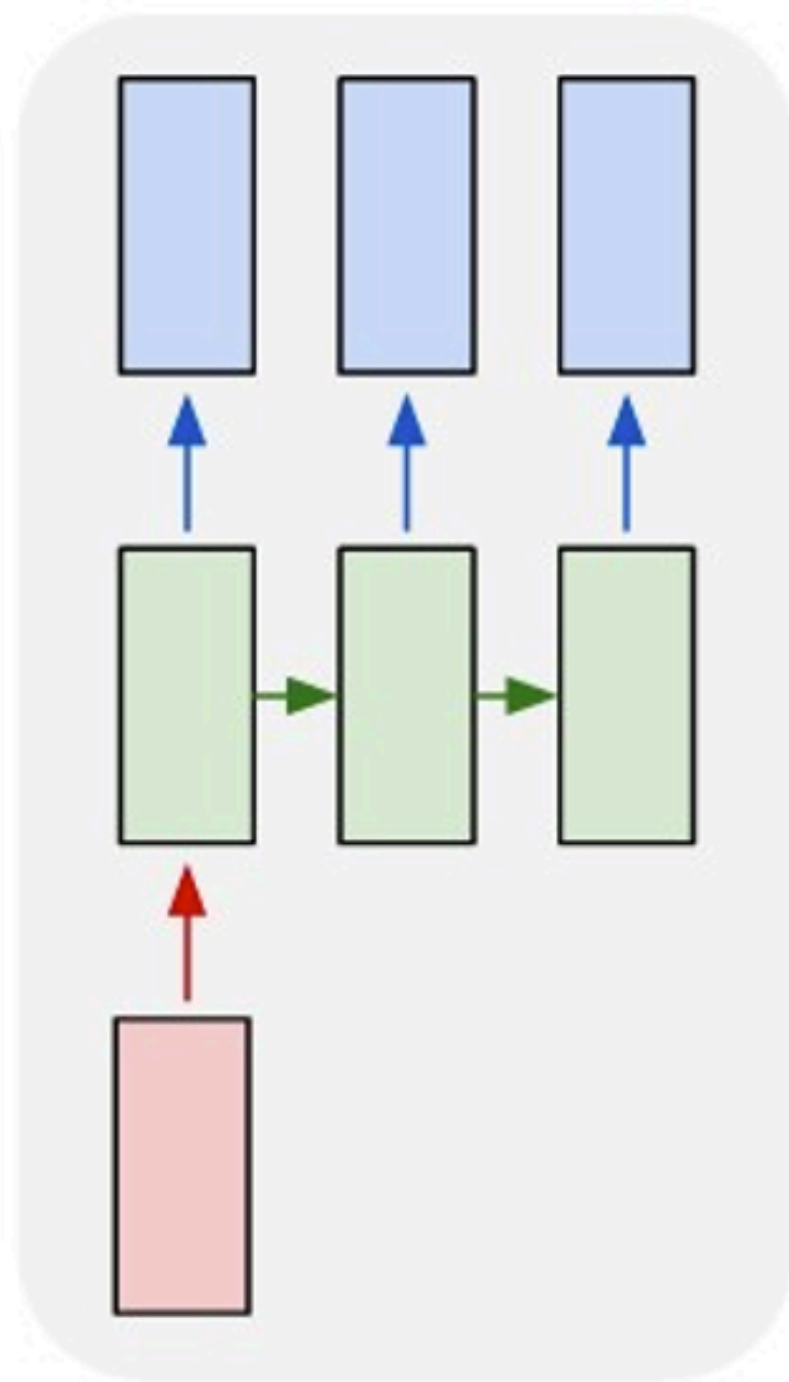
Types of sequence processing problems

one to one



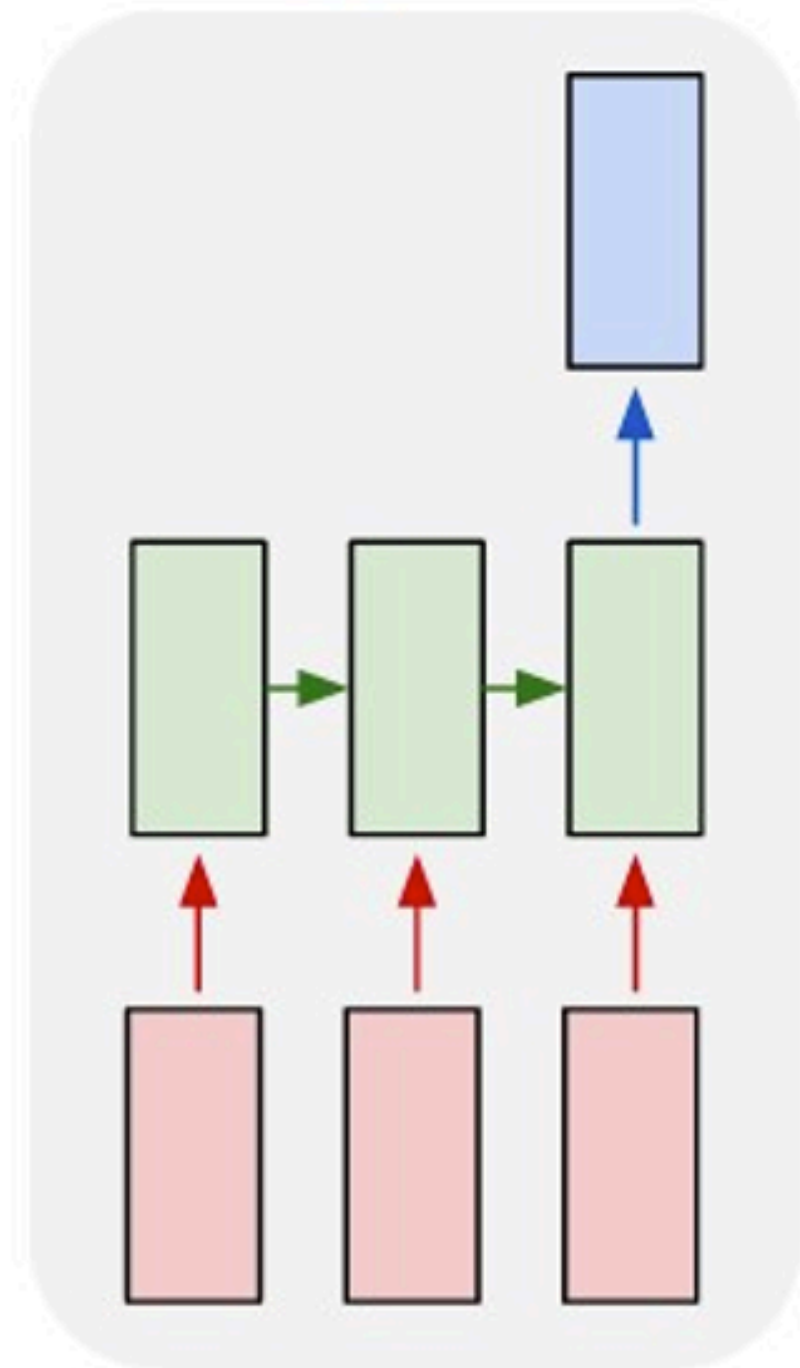
Vanilla
Neural
Networks

one to many



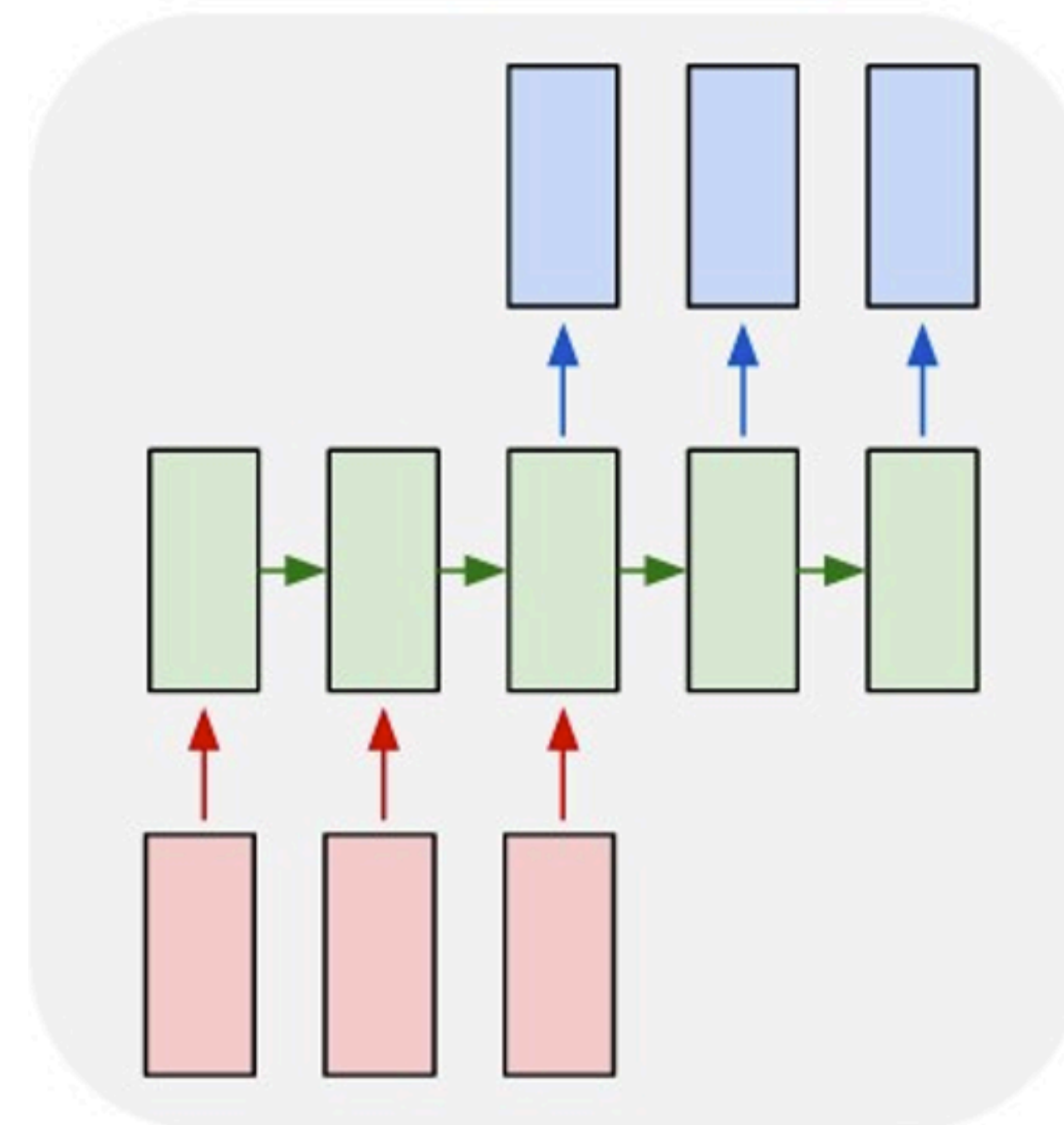
Text Generation

many to one



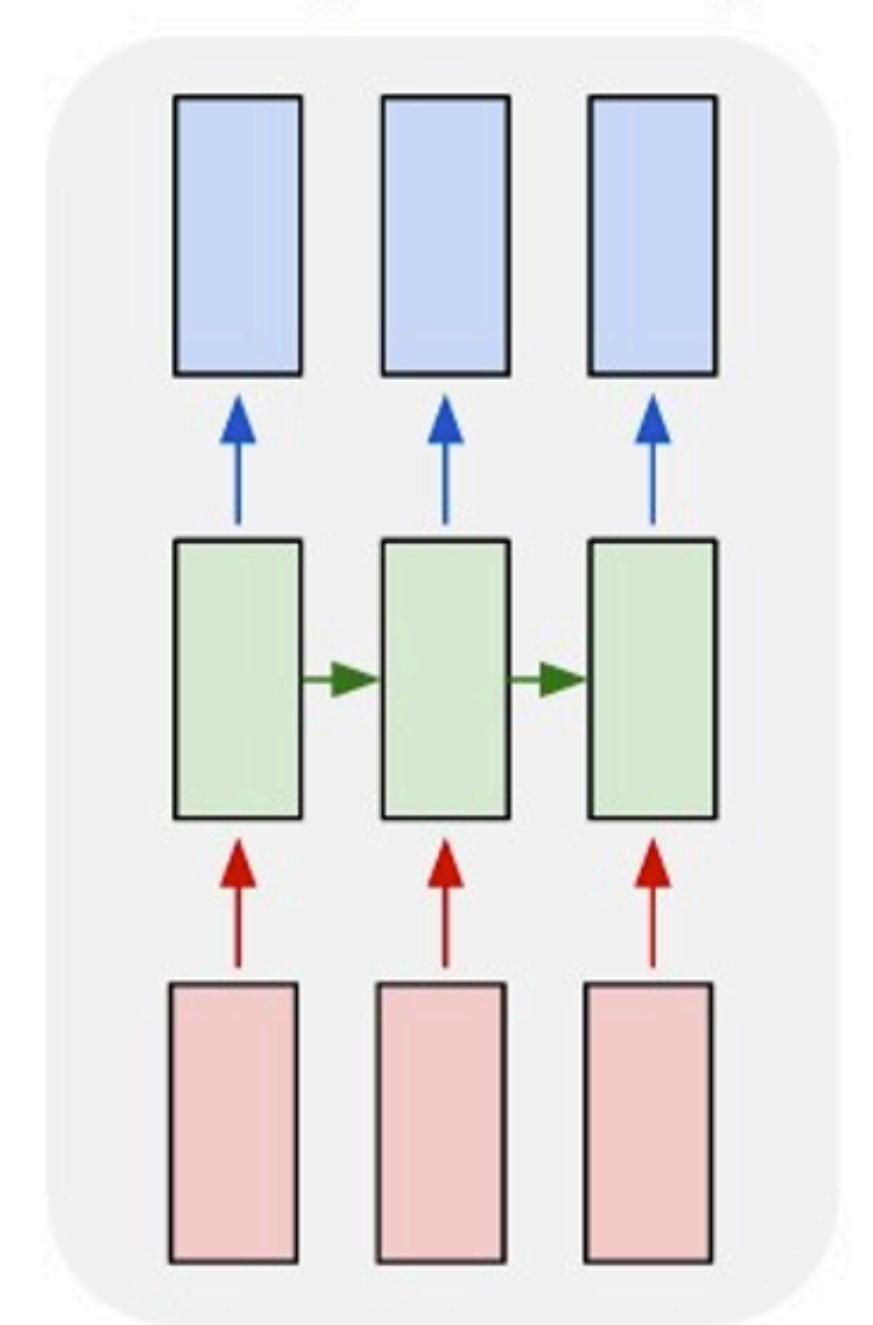
Text
Classification

many to many



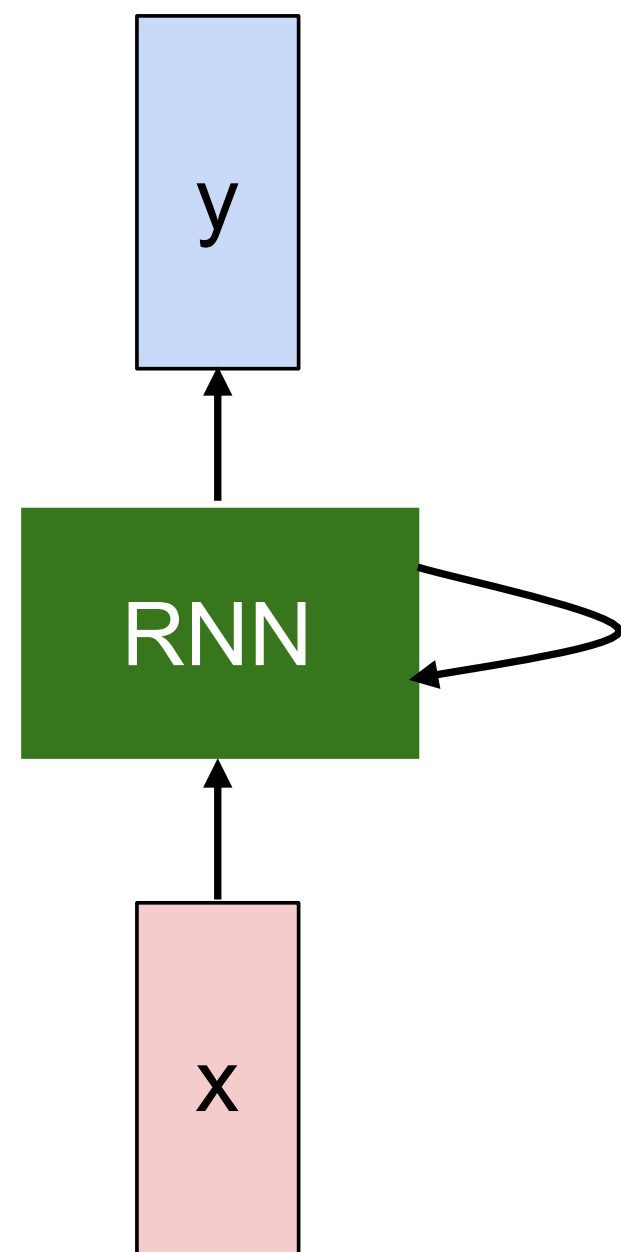
Neural Machine
Translation

many to many



Sequence
Tagging

Simple (vanilla) RNNs



$\mathbf{h}_0 \in \mathbb{R}^d$ is an initial state

function with weights W

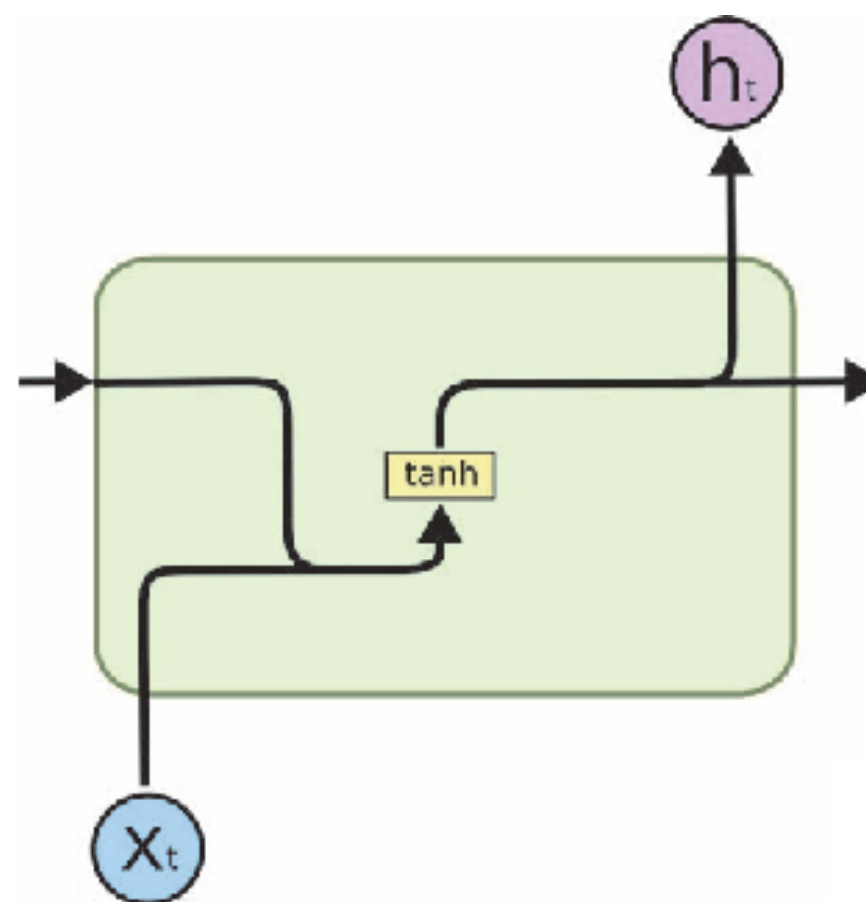
$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t) \in \mathbb{R}^d$$

new state old state input at time t

$\mathbf{h}_t$: hidden states which store information from $\mathbf{x}_1$ to $\mathbf{x}_t$

Output label for each time step: Denote $\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{W}_o \mathbf{h}_t)$, $\mathbf{W}_o \in \mathbb{R}^{|L| \times d}$

Simple (vanilla) RNNs:



$$\mathbf{h}_t = g(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}) \in \mathbb{R}^d$$

g : nonlinearity (e.g. tanh),

$$\mathbf{W}_h \in \mathbb{R}^{d \times d}, \mathbf{W}_x \in \mathbb{R}^{d \times d_{in}}, \mathbf{b} \in \mathbb{R}^d$$

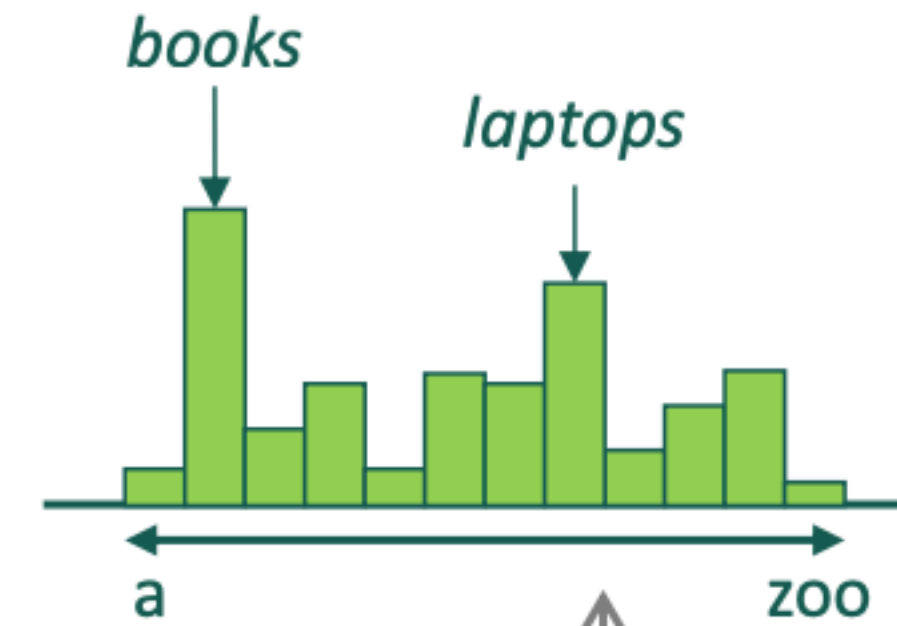
RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

Output label size: $|V|$

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$



hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

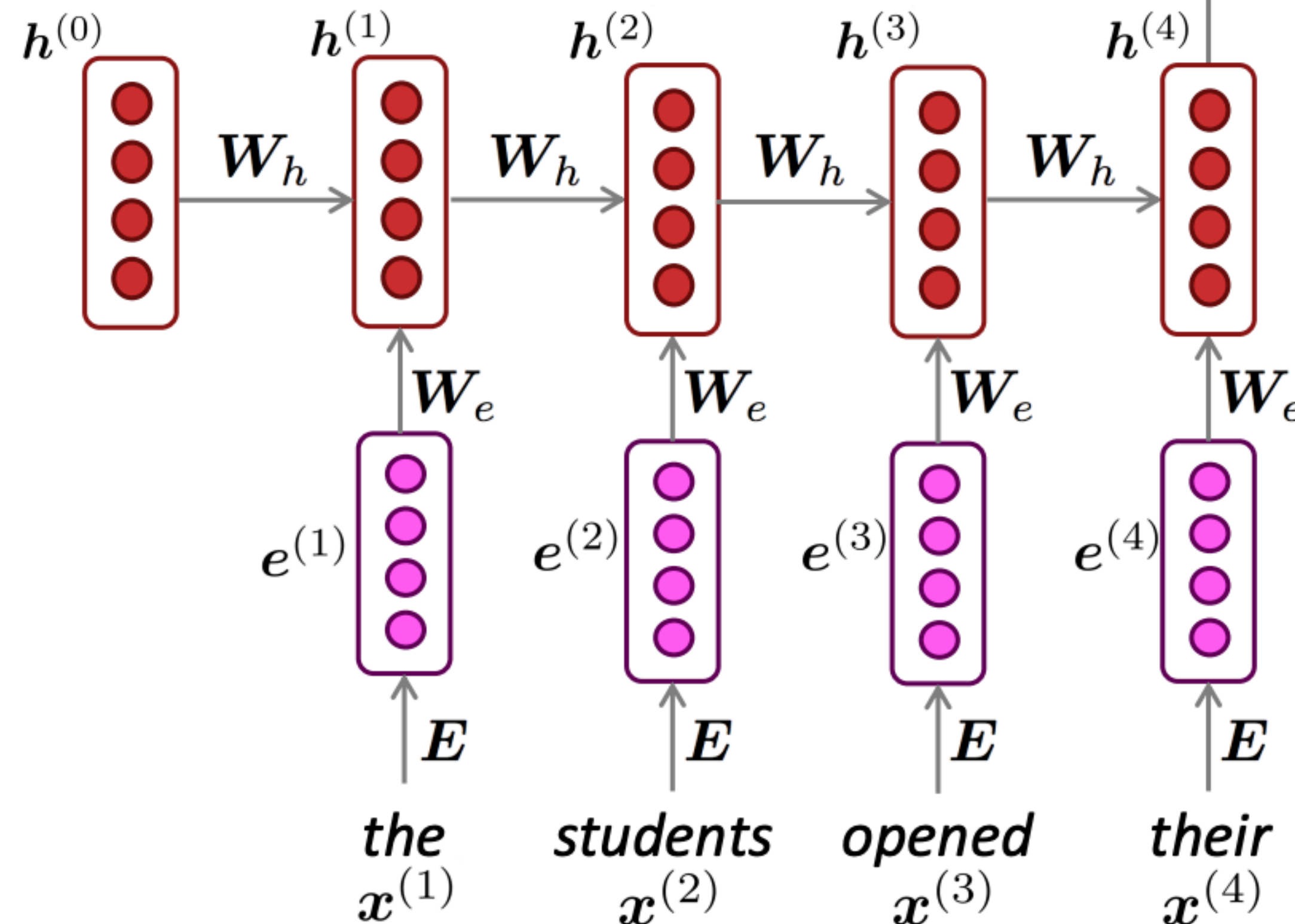
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Use word embeddings

RNNs: pros and cons

- **Advantages:**
 - Can process any length input
 - Computation for step t can (in theory) use information from many steps back
 - Model size doesn't increase for longer input context
 - Same weights applied on every timestep (symmetry in how inputs are processed)
- **Disadvantages:**
 - Recurrent computation is slow (can't parallelize) **Can parallelize with transformers!**
 - In practice, difficult to access information from many steps back

Progress on language models

On the Penn Treebank (PTB) dataset

Metric: [perplexity](#)

KN5: Kneser-Ney 5-gram

Model	Individual
KN5	141.2
KN5 + cache	125.7
Feedforward NNLM	140.2
Log-bilinear NNLM	144.5
Syntactical NNLM	131.3
Recurrent NNLM	124.7
RNN-LDA LM	113.7

$$\text{ppl}(S) = 2^x \quad \text{where}$$
$$x = -\frac{1}{W} \sum_{i=1}^n \log_2 P(S^i)$$

Training the RNN

Training an RNN Language Model

- Get a **big corpus of text** (sequence of words $x^{(1)}, \dots, x^{(T)}$)
- Feed into RNN-LM and compute output distribution $\hat{y}^{(t)}$ for **every step t** (i.e. predict for every word, given words so far)
- Loss function on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$)

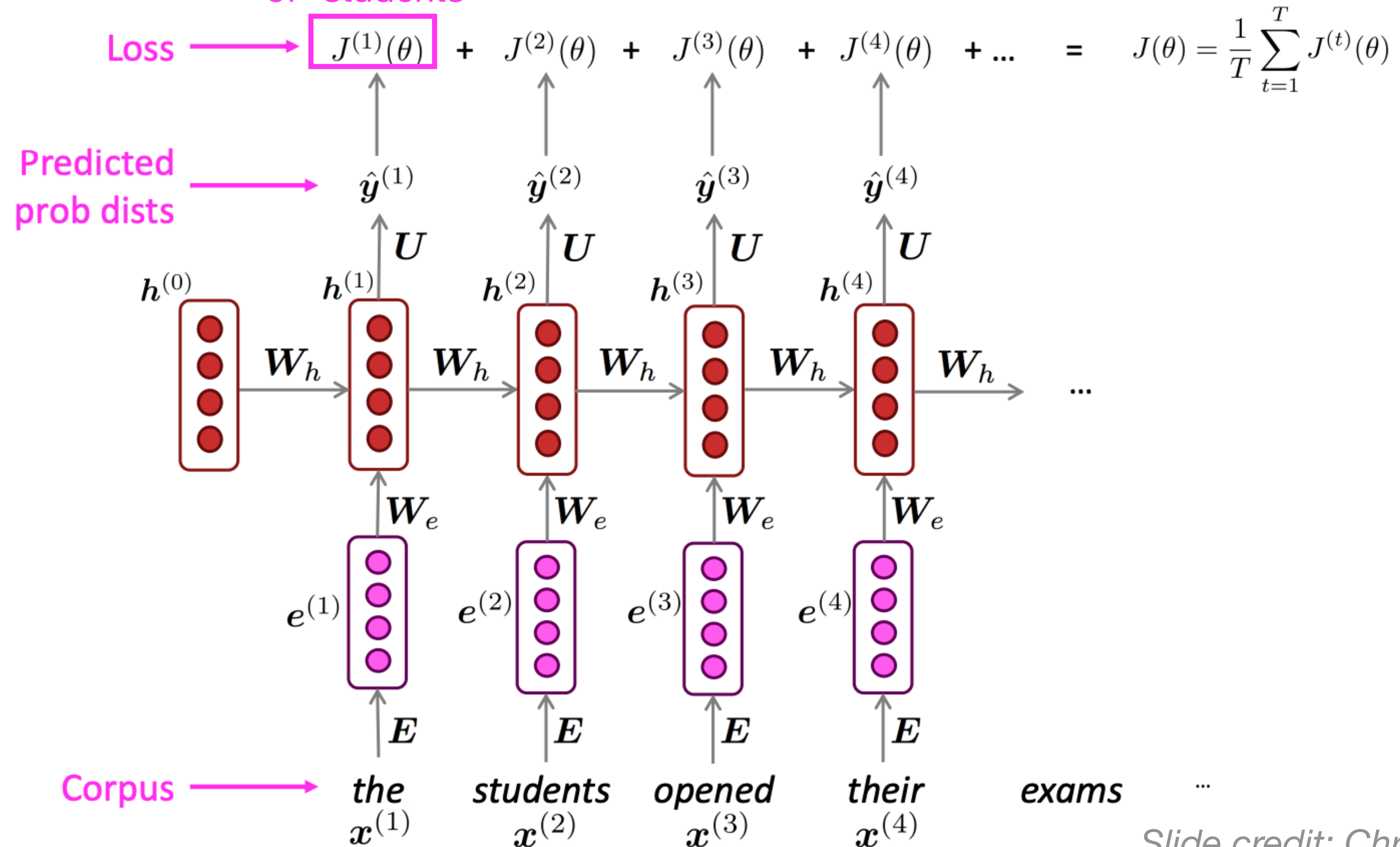
$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

- Average to get overall loss for the entire training set

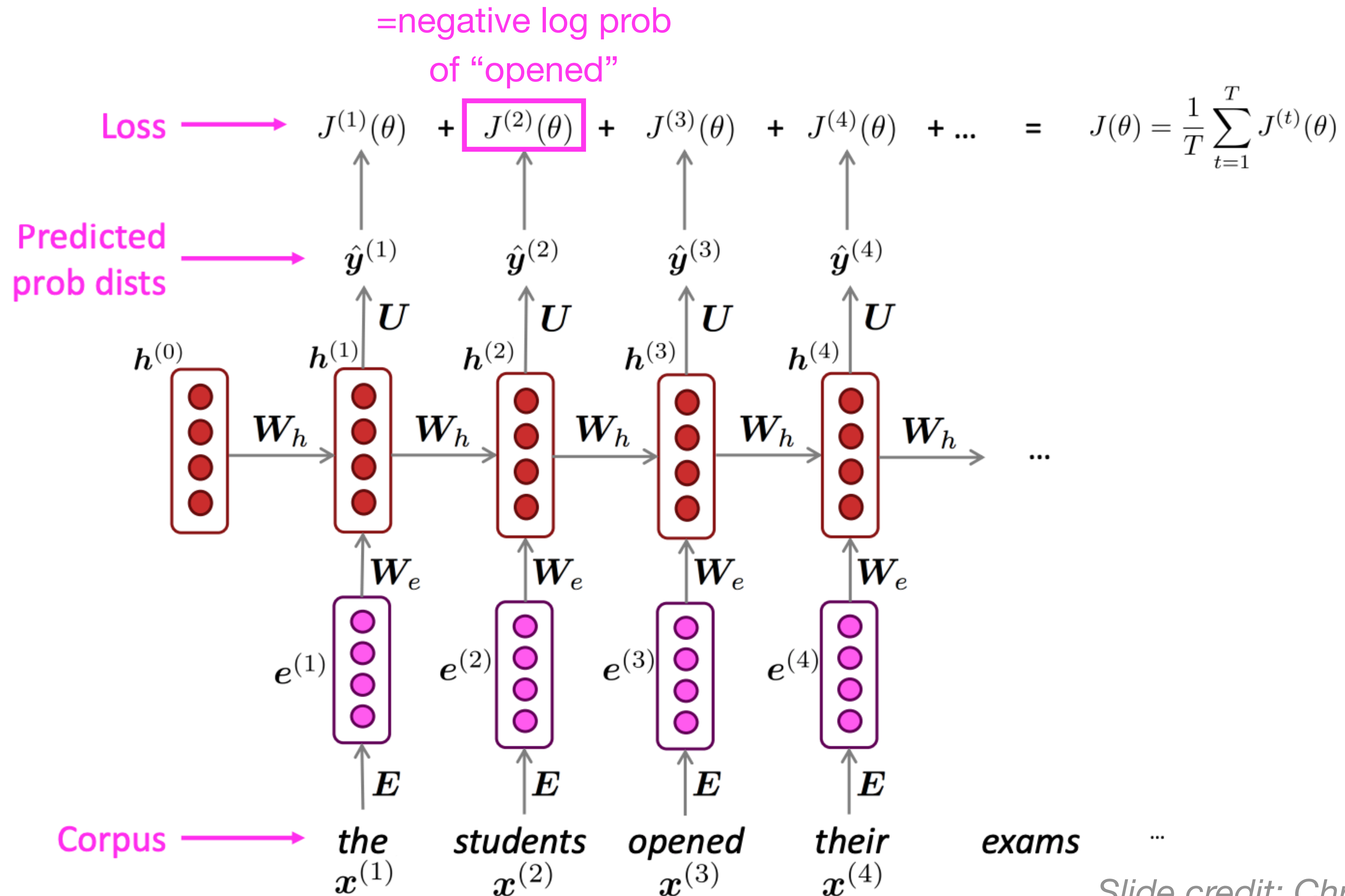
$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = - \frac{1}{T} \sum_{t=1}^T \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

Training an RNN Language Model

=negative log prob
of “students”



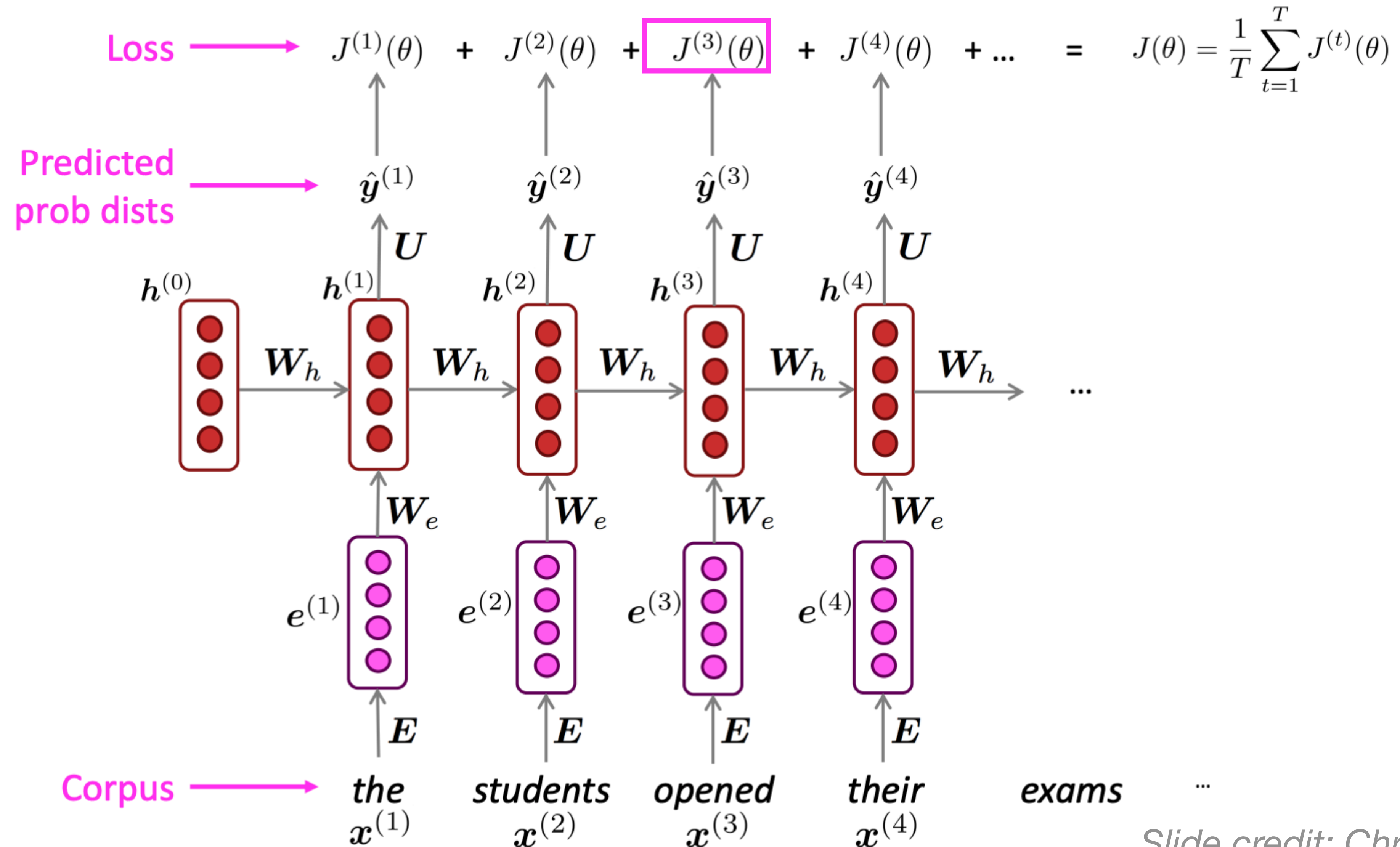
Training an RNN Language Model



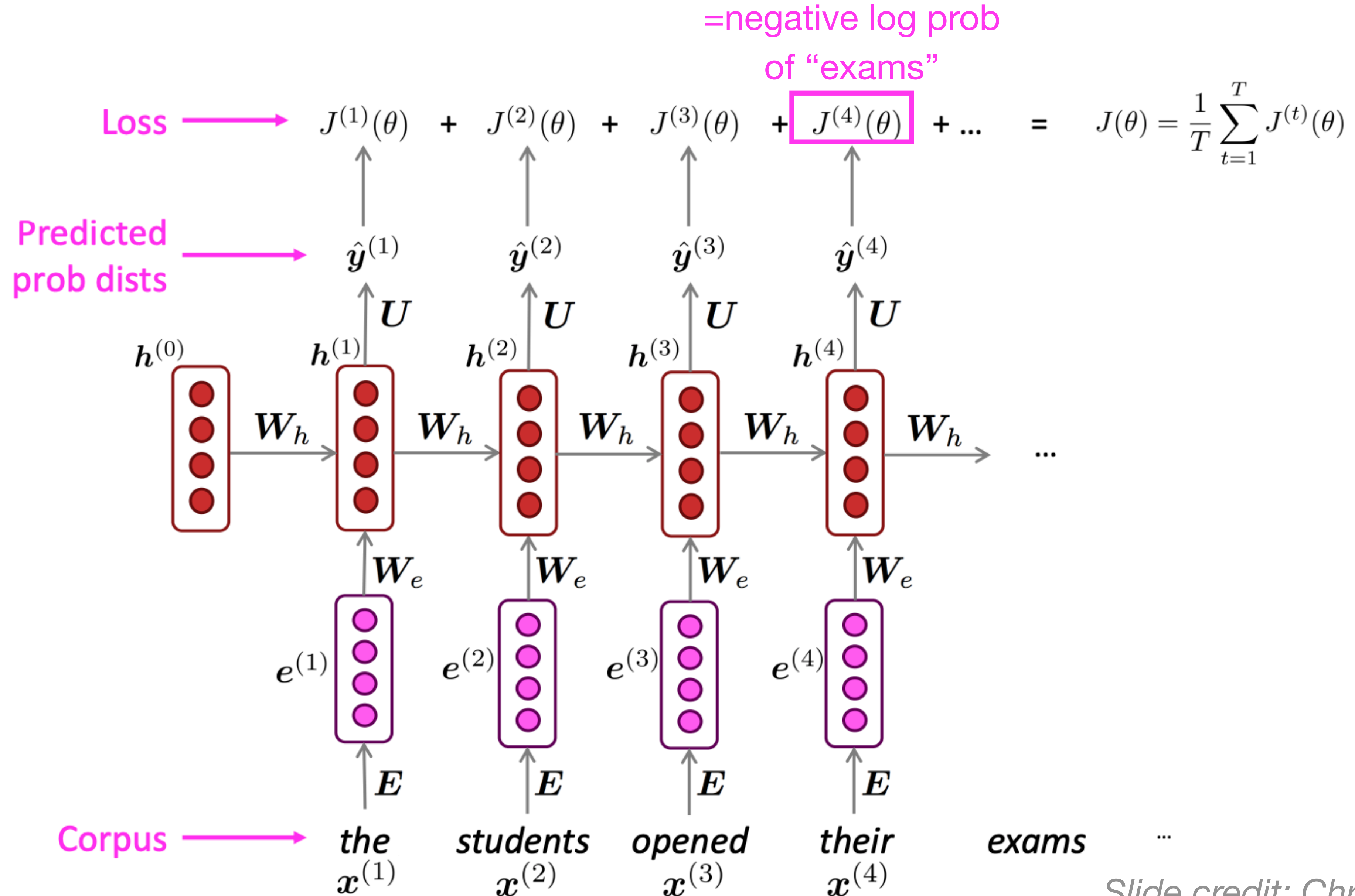
Training an RNN Language Model

=negative log prob

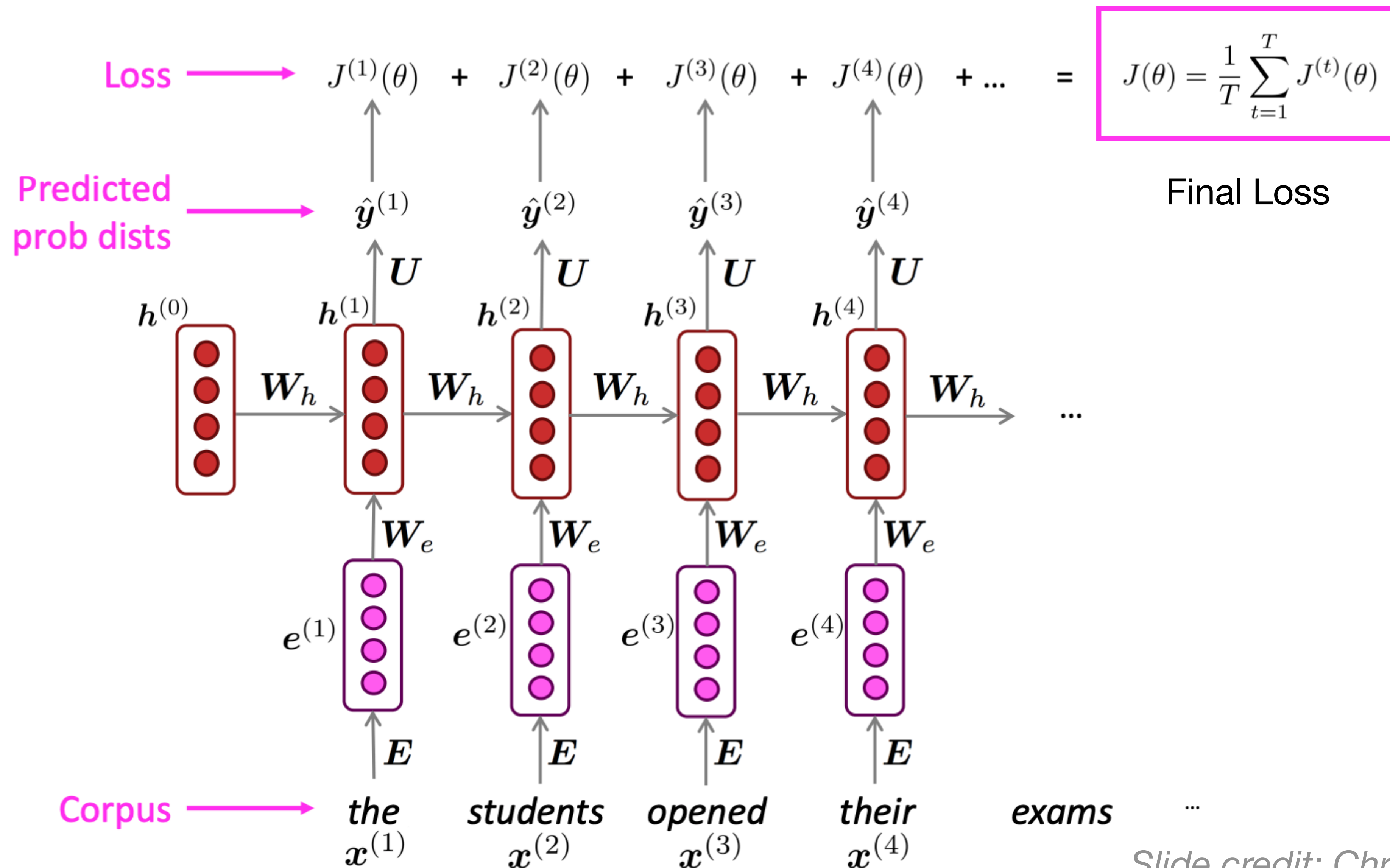
of “their”



Training an RNN Language Model



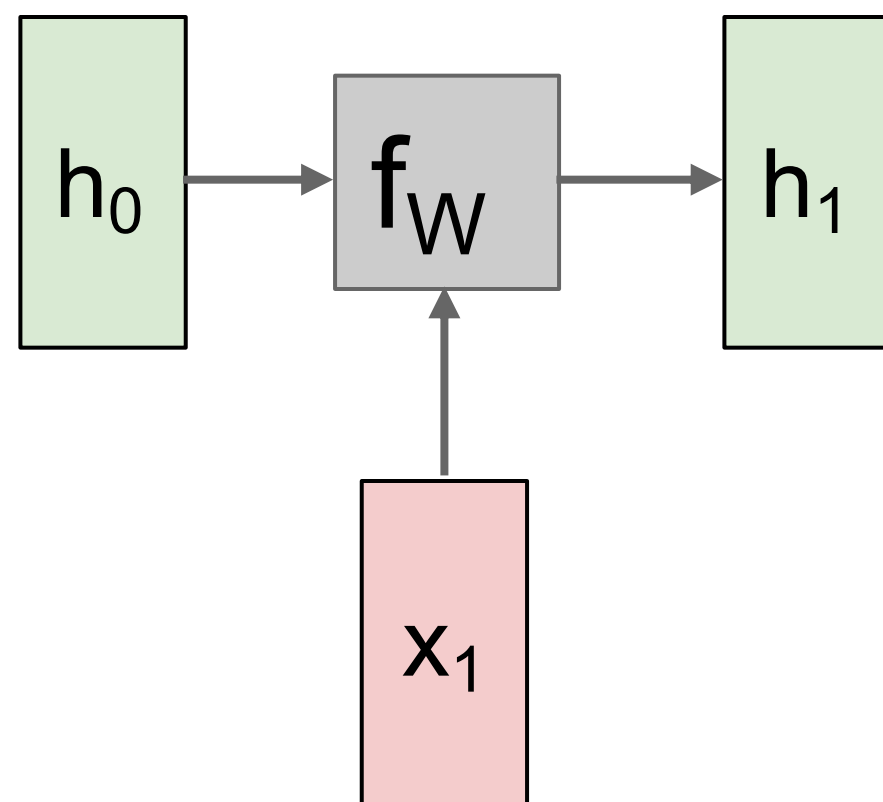
Training an RNN Language Model



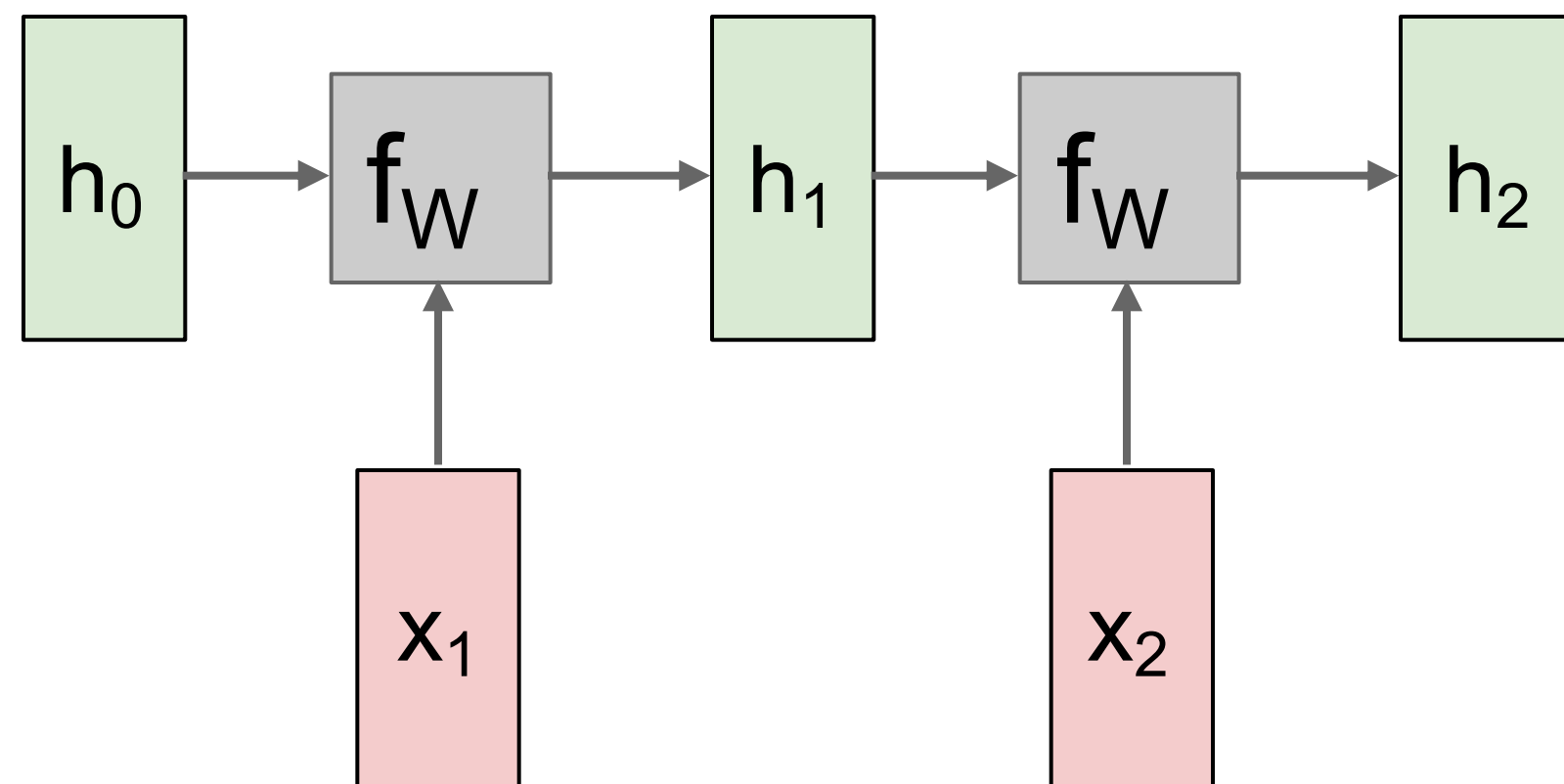
Training an RNN language Model

- Note that computing loss and gradients for the whole corpus at once is too expensive
- In practice, consider $x^{(1)}, \dots, x^{(T)}$ for a sentence (or a document)
- Use batching to parallelize computation over sentences
- Use SGD to estimate parameters
- Use computation graph with backprop

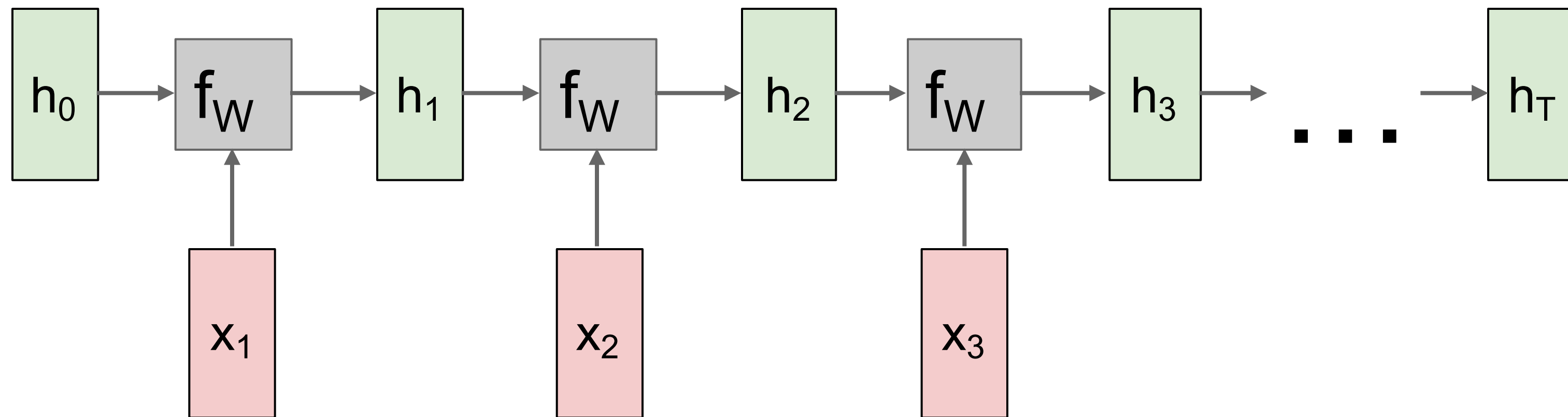
RNN Computation Graph



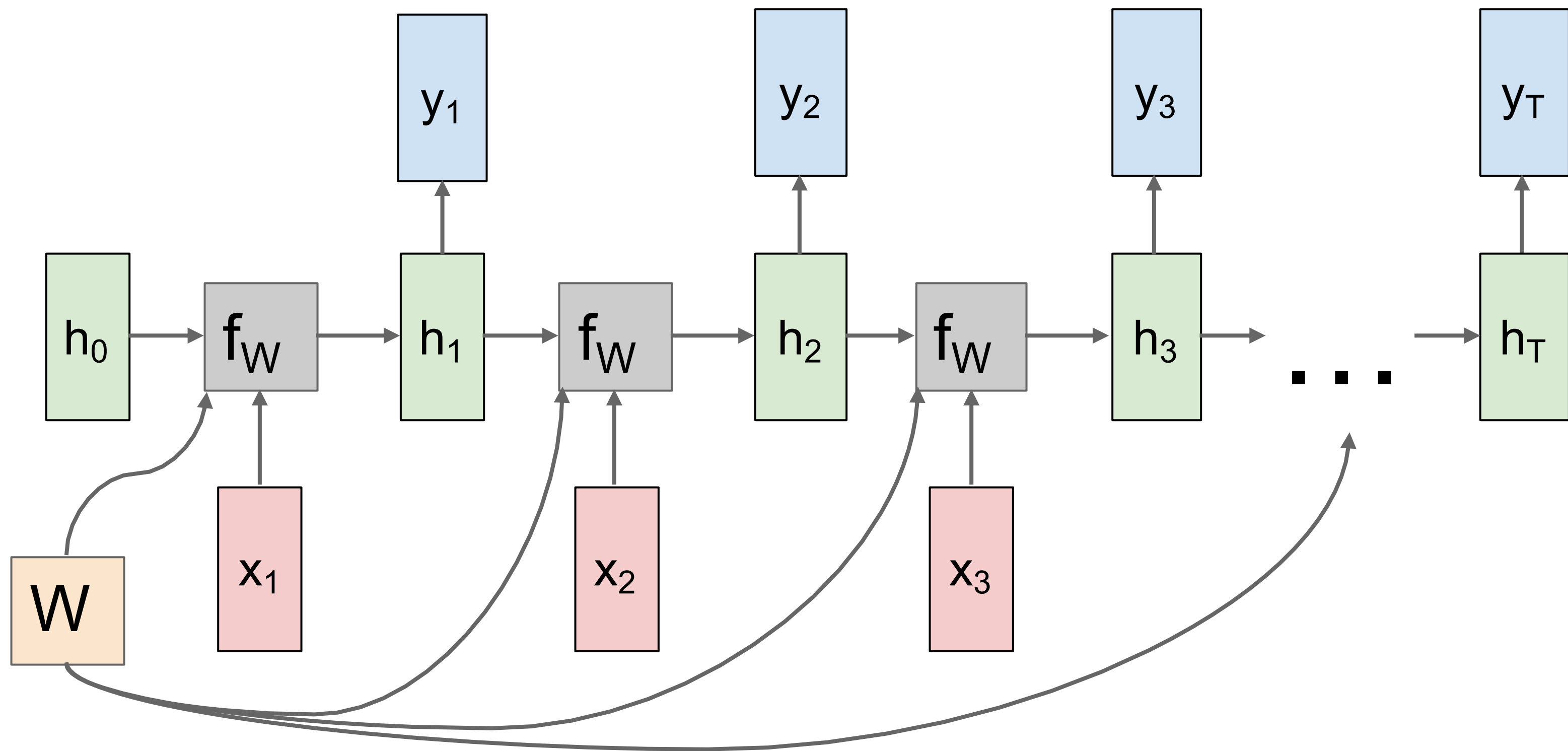
RNN Computation Graph



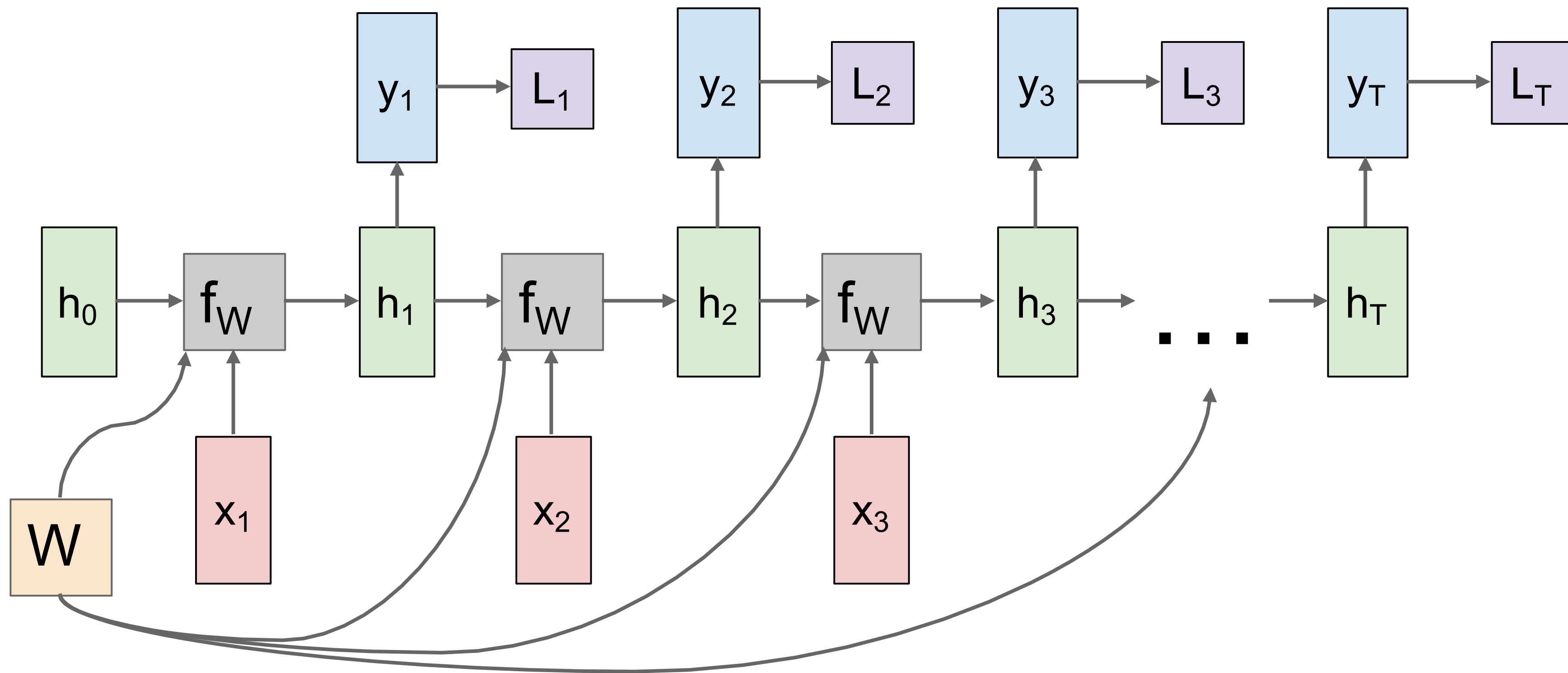
RNN Computation Graph



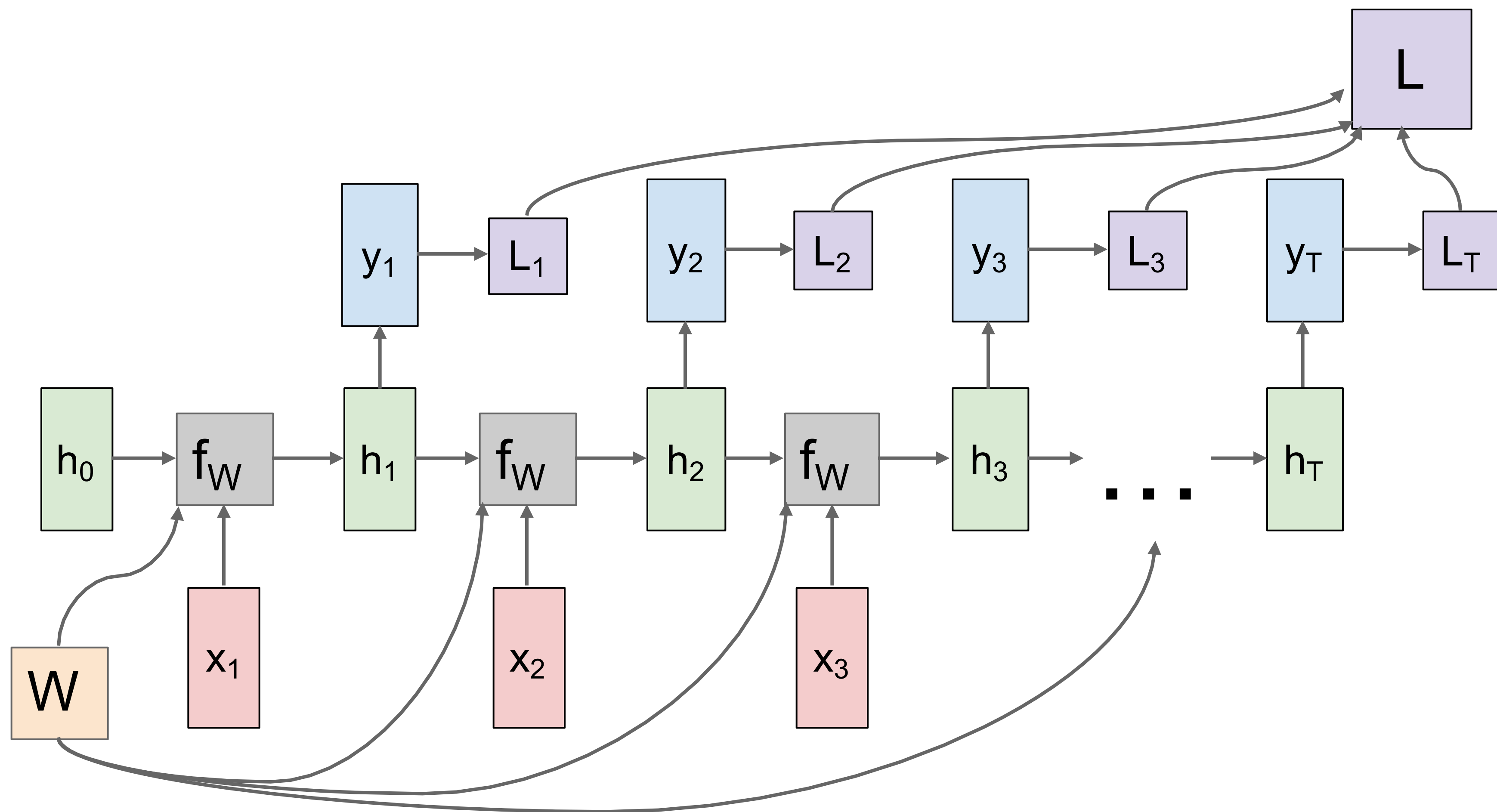
RNN Computation Graph



RNN Computation Graph

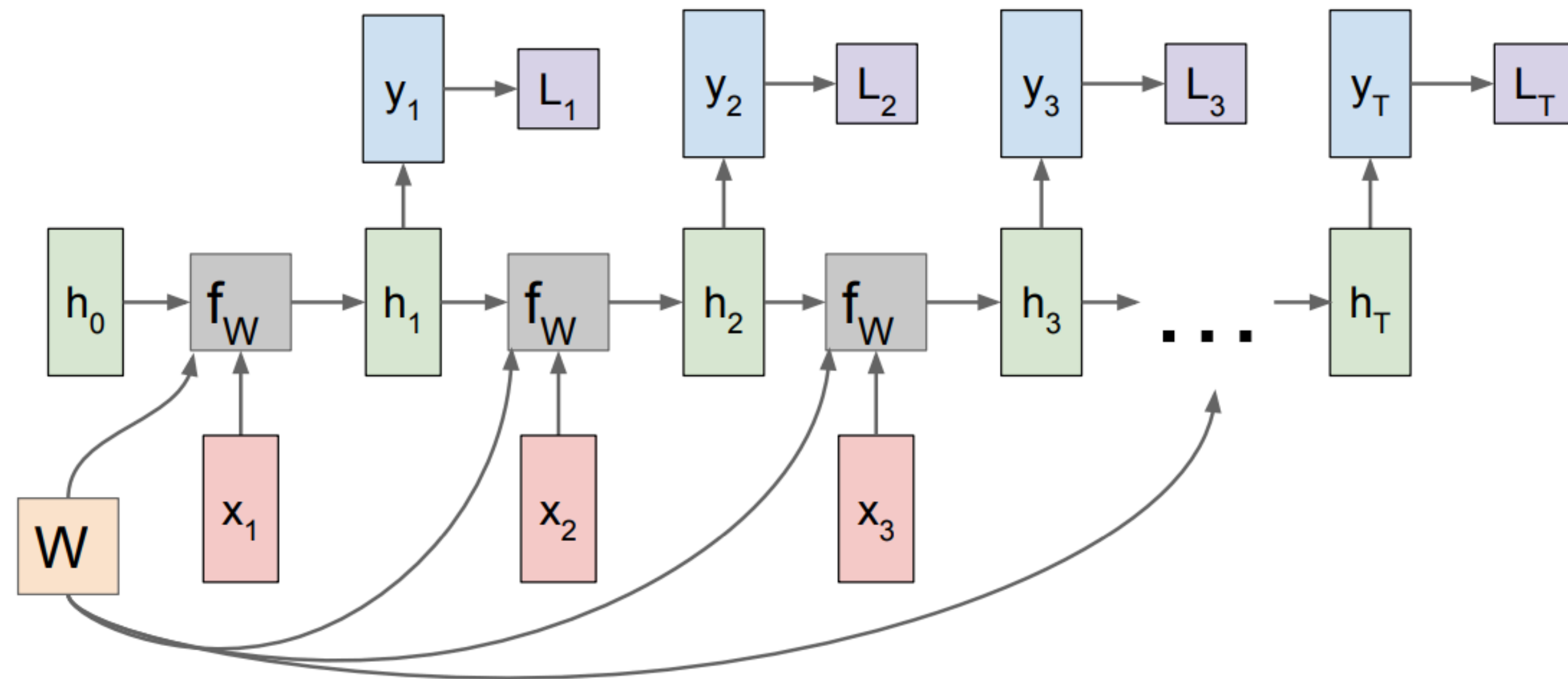


RNN Computation Graph



Training RNNLMs

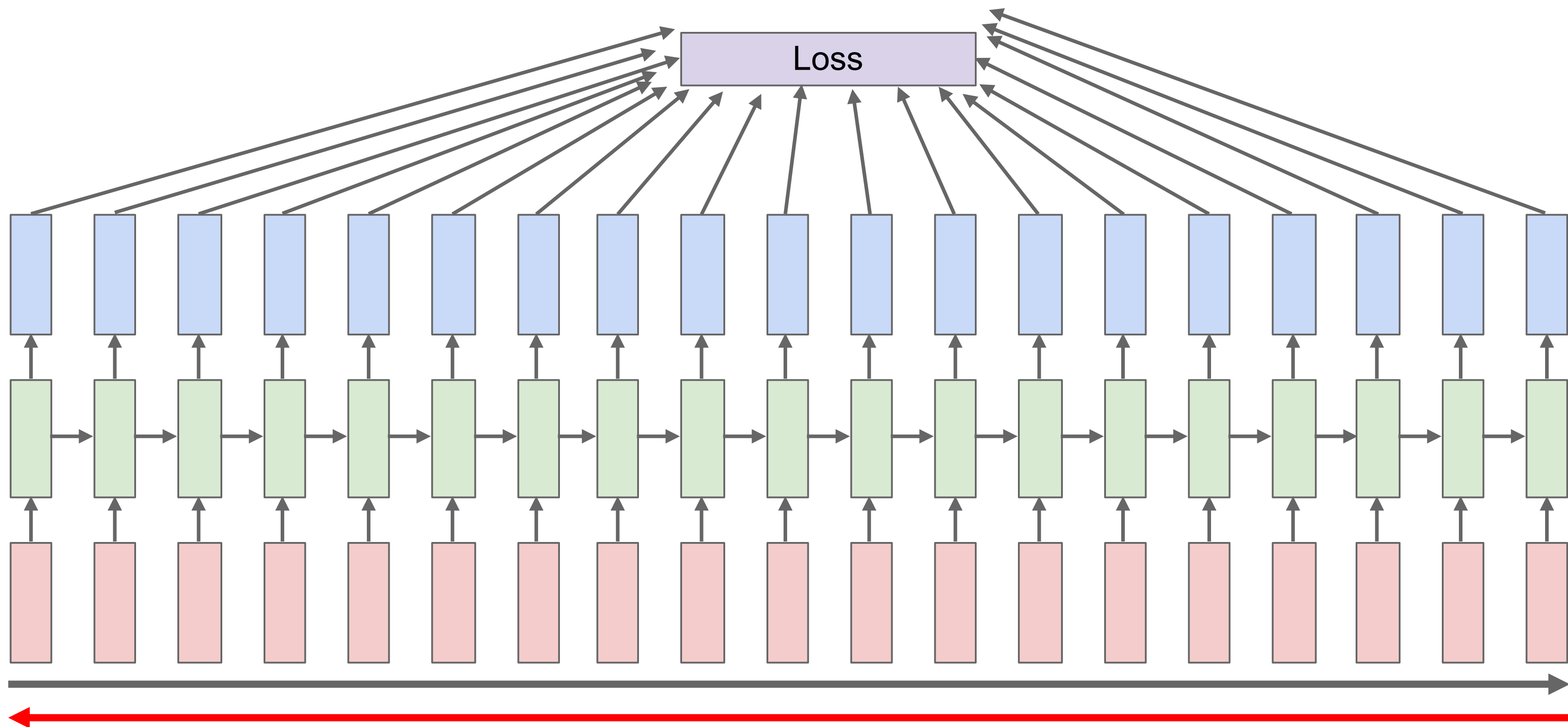
- Backpropagation? Yes, but not that simple!



- The algorithm is called Backpropagation Through Time (BPTT).

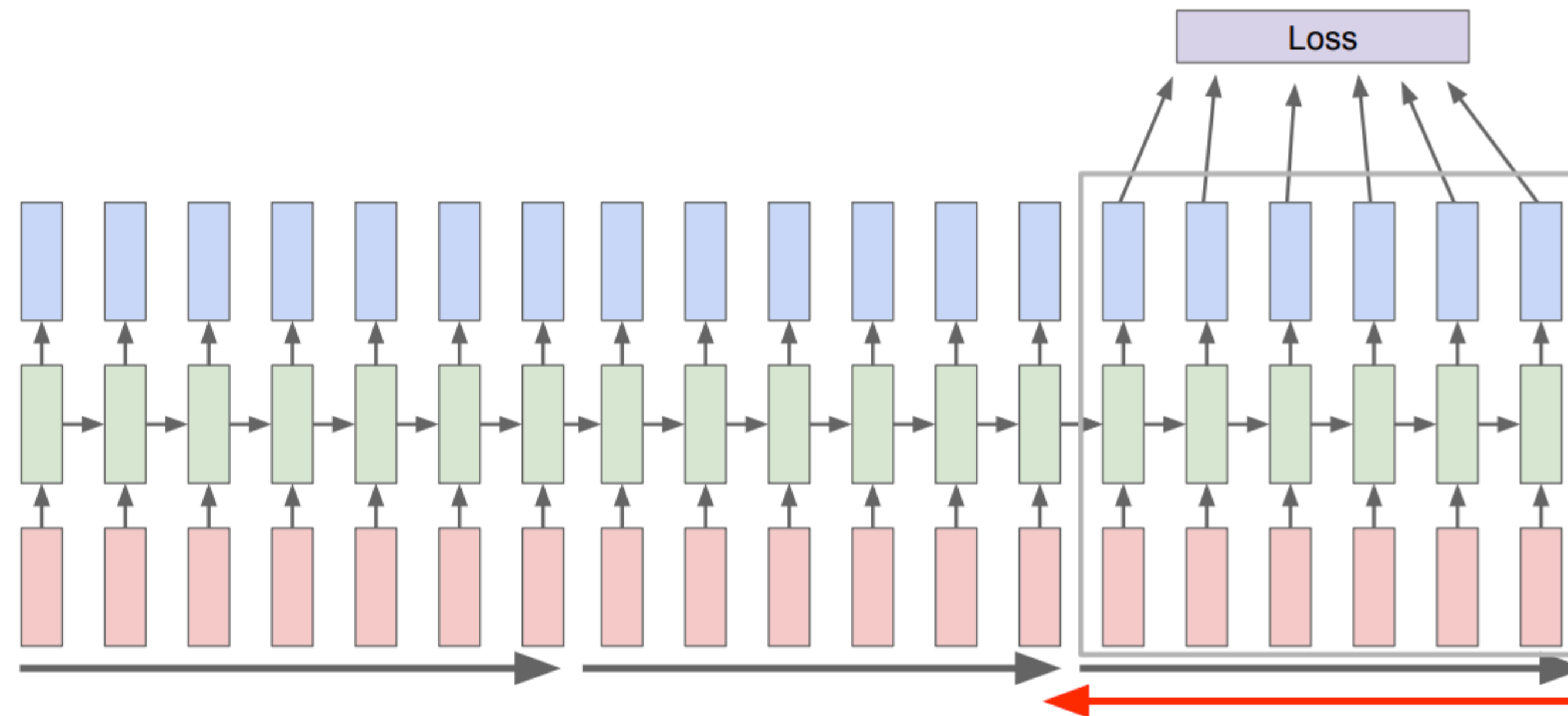
Backpropagation through time

Forward through entire sequence to compute loss, then
backward through entire sequence to compute gradient



Truncated backpropagation through time

- Backpropagation is very expensive if you handle long sequences



- Run forward and backward through chunks of the sequence instead of whole sequence
- Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

